Application Security Engineering — Study Plan

Independent Self-Study — Kyle Miskell — kmiskell@protonmail.com

0	I	II	III	IV	V	VI	VII	VIII	IX
Pre-Study	AppSec Intro	Web & Network	Core Vulns	Testing & Tooling	Cloud Security	Code Review	Sec+ Cert	Interview Prep	First Days

Contents

What Is Application Security? 3

Why AppSec Engineering 4

– The Case for This Path
– Your Structural Advantage

Why This Moment 5

Target Role: Application Security Engineer 6

– What It Is — Conceptually
– What It Is — In Practice
– Day-to-Day Responsibilities
– Philadelphia & Remote Market

Plan Schedule Overview 7

– Phase Timeline

Study Methodology 8

– Local & Regional Events
– Note-Taking
– Portfolio: Writeups & Project Site

AppSec: Potential Pitfalls & Risk Assessment 9

Study Plan Phase Summary 10

0 — Pre-Study: Foundations 12

– Learning Resources
– Schedule
– Projects & Writings
– Concepts and Their Use on the Job

I — AppSec Intro: A Dip into Offense 15

– Learning Resources
– Schedule
– Projects & Writings
– Concepts and Their Use on the Job

II — Web & Network Foundations 18

– Learning Resources
– Schedule
– Projects & Writings
– Concepts and Their Use on the Job

III — Core Vulnerability Classes 25

– Learning Resources
– Schedule
– Projects & Writings
– Concepts and Their Use on the Job

IV — Security Testing & Tooling 32

– Learning Resources
– Schedule
– Projects & Writings
– Concepts and Their Use on the Job

V — Cloud Security Fundamentals 41

– Learning Resources
– Schedule
– Projects & Writings
– Concepts and Their Use on the Job

VI — Secure Code Review & Threat Modeling 46

– Learning Resources
– Schedule
– Projects & Writings
– Concepts and Their Use on the Job

VII — CompTIA Security+ Certification 50
– Learning Resources
– Schedule
– Projects & Writings
– Concepts and Their Use on the Job

VIII — Interview Preparation & Interviewing 53
– Interview Preparation

IX — Expected First Days on the Job 56
– What You Bring on Day One

Appendix 57
– Application Strategy
– Lab Notes Format
– Philadelphia AppSec Community

What Is Application Security?

Application security — AppSec — is the discipline of finding, fixing, and preventing security vulnerabilities in software. Where most of software engineering asks *does this work?*, application security asks *can this be broken?*

Software has an attack surface: input fields, API endpoints, authentication flows and dependencies are potential entry points for an adversary. AppSec engineers systematically map that surface, probe it for weaknesses, and work with development teams to close them — before attackers find them first.

The discipline spans the entire software development lifecycle. In the design phase, it means threat modeling — drawing data flows and asking what happens when an attacker controls each input. During development, it means secure code review — reading code the way an attacker reads it, looking for injection points, broken authentication, insecure data handling. In testing, it means static analysis (SAST) that reads source code for vulnerability patterns, dynamic analysis (DAST) that probes a running application, and manual penetration testing that combines both with adversarial creativity.

The OWASP Top 10 is the field’s shared vocabulary for the most critical web application security risks. SQL injection, broken access control, cross-site scripting, security misconfigurations, software supply chain failures, and other key risks. These vulnerabilities are not abstract academic concerns — they are the specific vulnerability classes that appear in real breaches against real companies, year after year. AppSec engineers treat them as a framework for understanding how systems fail, not as a list to memorize.

This plan targets web application security specifically — the attack surface that lives at the HTTP layer, in APIs, in authentication flows, in the browser.

Why AppSec Engineering

The Case for This Path

You spent five years building the attack surface that AppSec engineers protect. You know FastAPI microservices, React frontends, AWS infrastructure, Docker containers, legacy PHP codebases, and additional technologies that AppSec work asks you to reason about adversarially.

Your Structural Advantage

AppSec engineers tend to come from development or from security. You come from development, so you know how vulnerable patterns get written. The code patterns that lead to SQL injection, authentication bypasses, and cross-origin resource sharing (CORS) misconfigurations are patterns that developers encounter every day. This plan adds the adversarial view of those same patterns.

Transferable skills:

- **FastAPI microservices** — REST API security is a core AppSec domain.
- **Python** — Production Python carries over to AppSec tooling: Semgrep rules, scanning automation, and additional security utilities.
- **React frontends** — Client-side vulnerabilities (XSS, cross-site request forgery (CSRF), insecure storage) require understanding how the browser executes JavaScript. React experience provides a strong foundation for understanding client-side vulnerability patterns.
- **AWS, Docker, Kubernetes** — Cloud and container security are AppSec-adjacent and increasingly part of the role.
- **Legacy PHP, raw SQL, and NoSQL** — You have worked in legacy PHP, MySQL, and DynamoDB codebases, where SQL injection, insecure direct object reference (IDOR), and NoSQL injection are commonly found.
- **Architecture & domain modeling** — You designed microservices and domain-driven services with deliberate separation of concerns and trust boundaries. Threat modeling is that same structural thinking turned toward attack: mapping data flows and asking where trust is assumed.
- **Testing discipline** — You wrote unit, integration, and regression tests. Security testing builds on the same habits: defining inputs, expected behavior, and failure modes.

Why This Moment

The case for entering application security in 2026 rests on published data.

The breach problem is not going away.

IBM's Cost of a Data Breach Report (2025 edition) found that the average U.S. cost of a data breach reached a record $10.22 million in 2025, even as the global average fell slightly to $4.44 million driven by faster AI-assisted detection. The U.S. number is the highest ever recorded. Nearly two-thirds of the 600 organizations IBM studied said they were still recovering from their breach at the time of the report. Verizon's 2025 Data Breach Investigations Report found that exploitation of vulnerabilities — the class of attack that AppSec engineers specifically exist to prevent — was present in 20% of all breaches studied; the 2026 edition reports that vulnerability exploitation has overtaken stolen passwords as the leading way attackers get in.

The workforce gap is structural.

ISC2's 2024 Workforce Study estimated a global cybersecurity workforce gap of roughly 4.8 million people; its 2025 study published no gap estimate, reporting instead that the shortage has moved from headcount to skills. The U.S. Bureau of Labor Statistics projects 21% job growth for information security analysts from 2025 to 2035, much faster than the average for all occupations, with about 14,100 openings a year. For application security specifically, Mordor Intelligence reports a 15% annual shortfall of AppSec engineers in the United States through 2026, demand growing faster than the supply of qualified people to fill it.

The shortage is not evenly distributed. ISC2 noted in 2025 that skills shortages now outweigh headcount shortages, so the shortage is in specific skills, not headcount. In ISC2's 2025 AI Adoption Pulse Survey, 52% of practitioners expected AI to reduce the need for entry-level staff, because the time-consuming routine tasks that once trained juniors are the ones AI takes first. What it does not take is the judgment-intensive work: finding business logic flaws that automated scanners rarely detect, threat modeling a new microservice architecture, explaining a critical finding to a VP of Engineering in terms that result in a fix. That work requires a human who can read code, understand systems, and think adversarially. That is the work this plan prepares you for.

The AppSec market is expanding fast.

Straits Research values the global application security market at $12.71 billion in 2025 and projects $42.29 billion by 2034, about 14% annual growth. Static application security testing is its largest segment and grows at about 15% a year. Application Security Engineer postings often list pay of about $120,000 to $195,000 in the Philadelphia region, and about $143,000 to $200,000 nationally. Product Security Engineer postings grew 12% year over year in 2024, seventh among the ten fastest-growing security roles in CyberSN's 2025 job-market analysis.

> **The structural argument:** IBM's 2025 data shows that organizations with a high security skills shortage averaged $5.22 million in breach costs against $3.65 million for those with little or none. Companies are not hiring AppSec engineers because it is a nice-to-have. They are hiring because the cost of not having them is measurable, large, and growing.

Target Role: Application Security Engineer

What It Is — Conceptually

Web application development and application security share the same foundation: HTTP, APIs, authentication flows, browser security models, database queries. An endpoint is an AppSec concern, an authentication flow is a potential bypass, and a React component that renders user input is an XSS candidate. Developers who learn AppSec understand the systems being defended — they are adding an adversarial lens to knowledge they already hold.

The shift is perspective. Where a developer asks *does this feature work correctly?*, an AppSec engineer asks *what happens when an attacker controls this input?* The underlying knowledge of the system is shared. The adversarial lens is the discipline.

What It Is — In Practice

- **Secure code review** (builds on: pull request review) — Reading code to identify vulnerability patterns: SQL queries built by string concatenation, tokens stored in localStorage, endpoints missing authorization checks, user data rendered without sanitization.
- **Penetration testing / DAST** (builds on: integration testing) — Running Burp Suite against a staging environment, intercepting HTTP requests, modifying parameters, observing whether the application behaves securely.
- **SAST tooling** (builds on: CI/CD linting and automation) — Integrating Semgrep into the pipeline. Writing custom rules that catch company-specific vulnerability patterns. Triaging scan results: true positives from false positives.
- **Threat modeling** (builds on: architecture review) — Given a proposed system design, enumerate the ways it could be attacked. STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.
- **Vulnerability management** (builds on: bug triage) — Prioritizing findings by severity and business impact. Writing clear reports developers can act on. Tracking remediation. Verifying fixes.

Day-to-Day Responsibilities

- Secure code review on high-risk PRs and new features — auth, payments, file upload, API endpoints
- Running and maintaining SAST tooling in CI/CD — Semgrep rules, false positive triage, new rule development
- Manual penetration testing of web applications and APIs — Burp Suite, methodology-driven, OWASP-aligned
- Threat modeling new system designs — STRIDE analysis, data flow diagrams, risk documentation
- Vulnerability reporting and remediation tracking — clear writeups, severity ratings, developer liaison
- Security training and developer education — lunch-and-learns, secure coding guidelines, champions programs

- Dependency and supply chain security — software composition analysis (SCA) scanning, CVE monitoring, patch prioritization

Philadelphia & Remote Market

AppSec has strong remote availability — a large share of postings offer remote or hybrid flexibility. Local Philly demand is moderate but real, anchored by healthcare, financial services, and enterprise tech.

Plan Schedule Overview

This plan is measured, not estimated: at 25 study hours per week the schedule below runs 8–9 months, counted from its own step tags and the author's logged reading and lab rates. The weeks marked optional (exam route) and conditional (cleared work) are counted; without them the schedule is shorter.

Day Type	Days/Week	Hours	Notes
Long Study	5	5 hrs	Reading, labs, implementation, tool practice, note card review, writeups
Day Off	2	—	Rest days. Required for retention.

Phase Timeline

0	I	II	III	IV	V	VI	VII	VIII	IX
Pre-Study	AppSec Intro	Web & Network	Core Vulns	Testing & Tooling	Cloud Security	Code Review	Sec+ Cert	Interview Prep	First Days
Wks 1–3	Wk 4	Wks 5–8	Wks 9–15	Wks 16–22	Wks 23–25	Wks 26–28	Wks 29–32	Wks 33–36	Wk 37+

Timeline: 8 months is the realistic case; 9 months is the outer bound. The first 1–3 weeks are pre-study before security content begins. Applications begin in Phase VIII, after the line of ready: the core classes, the pipeline tooling, cloud, code review and threat modeling, and the proof projects. Security+ (Phase VII) is taken only if cleared or defence-contractor work is chosen.

Study Methodology

The methods below apply to every phase. Following them consistently is the difference between completing labs and actually owning the material.

Local & Regional Events

Studying is not only reading and labs — attending local and regional security events is part of the work, because practitioners share current techniques there. A few worth knowing: the OWASP Philadelphia chapter (an AppSec-specific group), Philly 2600 (meets the first Friday of each month), and OWASP Global AppSec USA (city changes yearly; 2026 is San Francisco). Start attending early in the plan. See the Appendix section "Philadelphia AppSec Community" for the full list of groups and conferences.

Note-Taking: Notes → Note Cards

Two outputs for every chapter or lab section: working notes during study, note cards derived from them afterward. Notes are for comprehension; note cards are for retention.

- **During study:** Detailed working notes in your own words. For security concepts: what the vulnerability is, why the code is exploitable, how an attacker uses it, how to fix it. Questions you cannot answer yet.
- **After each section:** Extract note cards. One concept per card. Front: vulnerability name. Back: what it is, how to identify it in code, how to fix it. If you cannot explain the fix without looking, you do not own the concept.
- **For each lab:** write lab notes for a lab that has none. When a lab that already has notes is assigned again, update them if the repeat exposed a gap.
- **Material in Phases II–V gets full note cards.**

Portfolio: Writeups & Project Site

AppSec portfolios are built differently from software engineering portfolios. They center on documented offensive work: vulnerability writeups, lab solutions, capture-the-flag (CTF) walkthroughs, threat model documents, and additional artifacts. A writeup shows someone finding a vulnerability, understanding why it exists, exploiting it, and explaining the fix.

- **Writeup format:** every writeup uses the write-up structure from Writing Pentest Reports. The GitHub version follows this structure; the LinkedIn version can be a narrative rewrite of it (800–1,200 words).
- **Target:** the writeups scheduled in Phases I–III, then the Juice Shop assessment report and the threat model. Quality over quantity.
- **GitHub:** Commit from Day 1 — the very first commit is this study plan and its build script. Keep everything in public repositories — the plan, notes, tooling-phase scripts, custom Semgrep rules, automation tools, and lab artifacts. Every README explains the methodology and what the artifact demonstrates — not just usage instructions.

AppSec: Potential Pitfalls & Risk Assessment

AI Replacing AppSec Engineers — Real but Limited

AI-assisted SAST tools (GitHub Copilot Autofix, Snyk Code, Semgrep with AI triage) are improving and will automate more routine vulnerability scanning over time. The roles being automated are the routine scan-and-report functions, not the manual code review, threat modeling, and developer consultation that AppSec engineers do. The plan spends its time on that manual work.

Certifications Are Not the Portfolio

Sec+ is a recognized baseline credential in enterprise and regulated hiring. Postings that require it are often cleared or defence-contractor roles, so the plan schedules it only if that work is chosen, and treats the portfolio as the evidence of skill.

Moderate Philly Market

Philadelphia's AppSec market is real but not as deep as New York, DC, or San Francisco. The mitigation is remote — a large share of AppSec postings offer remote or hybrid. Philadelphia is a base, not a constraint.

Scope Creep into Red Team

AppSec engineers frequently perform application-level penetration testing as part of the role — Burp Suite, manual web app testing, OWASP methodology. This is expected and built into the plan. The drift to avoid is into full red team or network penetration testing, which is a distinct specialization requiring different skills (Active Directory, network exploitation, infrastructure pivoting).

Net assessment: The risks are real. They do not negate the market opportunity. The plan as designed — hands-on lab work over certifications, manual security skill over automated tooling familiarity, full-stack deployment capability — is structured to be resilient to the downside scenarios above.

Study Plan Phase Summary

This plan turns a working software engineer into an application security engineer with DevSecOps depth over eight to nine months. Rather than front-loading theory, it follows a deliberate arc: start with engineering projects, practice offense early, then build depth and professional tooling on top of that foundation.

It opens with engineering projects: Python and FastAPI, HTTP, Docker, a working local lab (Phase 0). Phase I follows with an introduction to the attacker mindset, so security work is hands-on from the start. Phase II then covers web and network foundations at security depth.

The middle of the plan builds the offensive skill set: going deep on the core vulnerability classes — injection, XSS, broken authentication and access control, server-side request forgery (SSRF), and the rest — by exploiting each one hands-on in deliberately vulnerable labs (Phase III), then layering on the professional toolchain (Burp Suite, plus SAST, DAST, dependency and secrets scanning) and earning the Burp Suite Certified Practitioner certification, a hands-on web-exploitation credential (Phase IV). A thread of AI and large language model (LLM) security — prompt injection, output handling, tool-calling risk — runs through these phases as well.

The back half pivots toward the work an AppSec engineer actually does day to day — cloud security basics (Phase V) and secure code review and threat modeling (Phase VI) — rounded out by CompTIA Security+ only if cleared or defence-contractor work is chosen (Phase VII).

It closes with focused interview preparation (Phase VIII) and the expected first days on the job (Phase IX). Applications begin in Phase VIII, after the line of ready.

> **Phase 0 — Pre-Study: Foundations** (Wks 1–3). **Purpose:** build the FastAPI application, Docker project, and local lab that later phases audit and scan, and review HTTP and cookies. **Resources:** FastAPI docs, HTTP (Notes + MDN), Docker (Notes). **Achievement:** a layered FastAPI application with dozens of endpoints and a running local lab.
>
> **Phase I — AppSec Intro: A Dip into Offense** (Wk 4). **Purpose:** an introduction to the attacker mindset — how web vulnerabilities are found and exploited. **Resources:** *Real-World Bug Hunting* (Ch 1–5), PortSwigger Web Security Academy, TryHackMe, OWASP WebGoat. **Achievement:** hands-on exploitation in WebGoat and PortSwigger labs and the SQL injection essay published.
>
> **Phase II — Web & Network Foundations** (Wks 5–8). **Purpose:** deep HTTP, web, and authentication foundations plus Burp basics. **Resources:** *Alice and Bob Learn Application Security*, *WAHH*, Burp Suite docs, PortSwigger Authentication path. **Achievement:** Burp fluency, HTTP/auth/session depth, and the PortSwigger Authentication, JSON Web Token (JWT), and OAuth labs cleared.
>
> **Phase III — Core Vulnerability Classes** (Wks 9–15). **Purpose:** exploit the core web vulnerability classes hands-on at practitioner depth. **Resources:** PortSwigger topic paths and their apprentice/practitioner labs, *Hacking APIs*, the OWASP Top 10, and the OWASP Top 10 for LLM Applications. **Achievement:** practitioner labs across the OWASP classes and the OWASP LLM Top 10, plus write-ups labelled with their OWASP Top 10 categories.
>
> **Phase IV — Security Testing & Tooling** (Wks 16–22). **Purpose:** tooling fluency — Burp mastery, SAST, DAST, SCA, and secrets scanning — and the BSCP certification. **Resources:** Burp docs, PortSwigger BSCP practice exams, Semgrep, Bandit, ZAP, Snyk, TruffleHog, Gitleaks. **Achievement:** passed the Burp Suite

Certified Practitioner (BSCP) exam, completed a full Juice Shop security assessment, and secured a small LLM-backed API.

Phase V — Cloud Security Fundamentals (Wks 23–25). **Purpose:** AWS security basics — shared responsibility, IAM, and common misconfigurations. **Resources:** flaws.cloud, flaws2.cloud, AWS IAM docs. **Achievement:** every level of the flaws.cloud and flaws2.cloud AWS security challenges completed, with one combined writeup, and the Phase 0 container image scanned and hardened.

Phase VI — Secure Code Review & Threat Modeling (Wks 26–28). **Purpose:** core AppSec craft — manual secure code review and threat modeling. **Resources:** OWASP Code Review Guide, the OWASP Application Security Verification Standard (ASVS), *Threat Modeling: Designing for Security*. **Achievement:** a documented code review and a threat model project.

Phase VII — CompTIA Security+ Certification (Wks 29–32). **Purpose:** conditional: only for cleared or defence-contractor work, where the certificate must be held at hire; otherwise skipped. **Resources:** the *Get Certified Get Ahead: SY0-701* study guide and Professor Messer practice exams. **Achievement:** passed CompTIA Security+ (SY0-701).

Phase VIII — Interview Preparation & Interviewing (Wks 33–36). **Purpose:** interview preparation and application strategy. **Resources:** *Impractical Python Projects*, a cybersecurity interview question bank, GitHub Security Lab, and mock interview platforms. **Achievement:** a finished portfolio and active interviewing.

Phase IX — Expected First Days on the Job (Wk 37+). **Purpose:** start strong in the new sub-field and role. **Resources:** your accumulated notes and reference books. **Achievement:** a first-two-weeks plan.

0

Pre-Study: Foundations

Weeks 1–3 · Before Security Content Begins

Three parallel tracks before security content begins. The FastAPI track runs the length of the phase: working through the official documentation and building a layered application alongside it. The others are a few days each, run in parallel.

Learning Resources

- **FastAPI Official Documentation** — Official documentation for FastAPI, a web framework for building APIs with Python.
- **HTTP Notes** — Existing notes, reviewed and expanded in this phase. They cover request-response, methods, status codes, URL anatomy, headers, DNS, TCP/IP basics, and additional topics.
- **MDN Web Docs: Using HTTP Cookies** — Specifically for Secure, HttpOnly, and SameSite cookie attributes. Expands the existing HTTP notes.
- **Docker Notes** — Existing notes and note cards, reviewed in this phase. They cover containers, images, Dockerfiles, networking, volumes, Compose, Swarm, Stack, and additional topics.
- **DigitalOcean: Install and Use Docker** — A walkthrough of the container lifecycle.
- **OWASP WebGoat** — A deliberately insecure web application, maintained by OWASP, designed to teach web application security lessons.

Pre-Study Schedule — Weeks 1–3

01. SETUP: Git & GitHub Hygiene · Day 1

- Confirm the GitHub profile is public, the bio is current, and the location is set to Philadelphia, PA. Review existing pinned repos.
- Create a new repository for the FastAPI project. Commit as you build — clean messages, logical history.
- Set up LinkedIn in parallel: headline and About section state the transition into application security. Certifications and portfolio links get added as they are earned and built.

02. REVIEW: HTTP & Cookies · Days 1–3

- **HTTP:** Review existing HTTP notes and note cards. These cover the request-response model, methods (GET, POST, PUT, PATCH, DELETE), status codes (2xx, 3xx, 4xx, 5xx), URL anatomy, headers, DNS, TCP/IP basics, and additional topics.
- **Cookies:** Expand the existing HTTP notes with cookie security. Read MDN ‘Using HTTP cookies’ specifically for the security attributes: Secure (only sent over HTTPS), HttpOnly (inaccessible to JavaScript), and SameSite (controls cross-site request behavior). These three attributes appear throughout security review work.
- **SQL:** Do a fast review of joins, normalization, indexing, and additional topics using existing SQL notes and note cards.

03. READ + BUILD: FastAPI Documentation — Complete · Weeks 1–3

- The official docs cover Dependencies, Security, Middleware, CORS, Request Body, Response Model, and many additional sections.
- **Starting point:** Review existing FastAPI notes, then continue through the remaining tutorial sections. Take notes and extend the project alongside each section.
- **Build a FastAPI application alongside the reading:** a layered, domain-driven REST API with dozens of endpoints across several domains. Give it OAuth2 with JWT, scoped authorization through FastAPI's Security module, dependency injection, a SQL database, middleware, CORS, tests, and additional features.
- Containerize and deploy it. Leave some security weaknesses in deliberately, so it can serve as an audit and scanning target from Phase II onward.

04. LEARN + PRACTICE: Docker · Days 2–4

- Review existing Docker notes and note cards: containers, images, networking, Dockerfiles, volumes, Compose, Swarm, and additional topics.
- Work through DigitalOcean's How to Install and Use Docker guide for a walkthrough of the container lifecycle.
- **Practice:** pull and run OWASP WebGoat in a container — publish it to a port, open it in the browser, then stop and remove it. Running a vulnerable target app in a container is a Docker skill this plan uses repeatedly: later phases run Juice Shop, DVWA, and additional target apps the same way.
- **Build a Docker review project:** a command script that exercises images, containers, networks, volumes, and additional Docker features, plus a Dockerfile and a Compose file that serve a simple web application.

Projects & Writings — Pre-Study

Project: FastAPI Application

A layered, domain-driven REST API with dozens of endpoints, OAuth2 with JWT, scoped authorization, a SQL database, tests, and additional features. Containerized, deployed, and pushed to GitHub.

Project: Docker and Docker Compose Review

A command script covering images, containers, networks, volumes, and additional Docker features. A Dockerfile that serves a FastAPI application with Uvicorn as a non-root user, and a Compose file with live reload. Pushed to GitHub.

Concepts and Their Use on the Job

FastAPI security internals

How FastAPI implements authentication and authorization via the Security module and Depends(). Where JWT validation happens. How middleware intercepts requests. How CORS is configured at the framework level.

On the job: These are the layers you audit when reviewing application code for security issues.

Cookie security attributes

Secure, HttpOnly, and SameSite are not just configuration options — they are security controls.

On the job: A session cookie set without Secure, HttpOnly, or SameSite is reported as a finding in a security review.

Docker as a tool in AppSec

Containers package, isolate, and deploy applications, among other uses, plus they also create attack surfaces: exposed ports, over-permissioned images, secrets in environment variables, and additional risks.

On the job: Running a target app in a container, as with WebGoat here, is one way to set up an assessment environment. The local target apps in this plan run the same way.

I

AppSec Intro: A Dip into Offense

Week 4

This phase exploits several vulnerability classes hands-on before Phase II and practices the attacker's question: what happens when an attacker controls this input?

Learning Resources

- **PortSwigger Web Security Academy** — Free, online web security training from the creators of Burp Suite, with interactive labs.
- **OWASP WebGoat** — A deliberately insecure web application, maintained by OWASP, designed to teach web application security lessons.
- **TryHackMe** — Browser-based, guided rooms. This phase uses four: HTTP in Detail, Burp Suite: The Basics, Vulnerabilities 101, and Writing Pentest Reports.
- **"*Real-World Bug Hunting*"** — Peter Yaworski, No Starch Press, 2019 — An introduction to web vulnerabilities through disclosed bug bounty reports, organized by vulnerability class.
- **Understanding CVSS Base Scores** — An article from Safe Security that defines the CVSS base metrics and their values.
- **CVSS v3.1 Calculator** — The scoring calculator from FIRST, the organization that maintains CVSS.

AppSec Intro — Week 4

01. EXPLORE: TryHackMe Orientation · Week 4

- Complete two TryHackMe rooms — HTTP in Detail and Burp Suite: The Basics
- Study them for orientation, not mastery

02. EXPLORE: PortSwigger Academy Orientation · Week 4

- Create a free PortSwigger account
- Skim the SQL injection material to get oriented, then complete the following labs:

  Apprentice:
  - SQL injection vulnerability in WHERE clause allowing retrieval of hidden data
  - SQL injection vulnerability allowing login bypass

  Practitioner:
  - SQL injection attack, querying the database type and version on Oracle
  - SQL injection attack, querying the database type and version on MySQL and Microsoft
- Skim the cross-site scripting material to get oriented, then complete the following labs. Apprentice: Reflected XSS into HTML context with nothing encoded and Stored XSS into HTML context with nothing encoded
- This phase is a quick survey of how these attacks work, not an attempt at mastery. Use the official solutions where needed. Phases II–III return to these classes in depth.

03. READ: "*Real-World Bug Hunting*" Ch. 1–5 · Week 4

- Chapter 1 covers bug bounty and HTTP basics; Chapters 2–5 are each a vulnerability class — open redirect, HTTP parameter pollution, CSRF, and HTML injection / content spoofing — illustrated with real disclosed bug bounty reports

04. LEARN: Writing Pentest Reports · Week 4

- Complete the free TryHackMe room Vulnerabilities 101. Its Task 3, Scoring Vulnerabilities (CVSS & VPR), introduces CVSS and its severity ratings
- Read Understanding CVSS Base Scores, which defines the CVSS base metrics and their values. Then, score the vulnerability from the SQL injection vulnerability allowing login bypass lab with the CVSS v3.1 calculator
- Complete the free TryHackMe room Writing Pentest Reports, which teaches the write-up structure used for the writeups in this plan

05. PROJECT: WebGoat — Hands-On Exploitation · Week 4

- Ensure local WebGoat still executes as expected in local Docker
- Complete these WebGoat lessons: SQL Injection (intro), Cross Site Scripting, Cross Site Scripting (stored), Authentication Bypasses, and Insecure Direct Object References (IDOR)
- For each lesson: exploit it, then write lab notes for it if none exist, with the root cause traced to the Java source
- Then write the essay 'What SQL Injection Taught Me About Trusting Input' (900–1,200 words): a plain-English account of what SQL injection is, why it works, and what surprised you in the WebGoat and PortSwigger labs. Publish on LinkedIn when polished

Projects & Writings — AppSec Intro

Writing: What SQL Injection Taught Me About Trusting Input (900–1,200 words)

Plain-English explanation of what SQL injection is, why it works, and what surprised you. Publish on LinkedIn when polished. Framing: 'I have been building APIs for five years. Here is what I learned when I tried to break one.'

Concepts and Their Use on the Job

The OWASP Top 10

A published list of web vulnerability categories, introduced in this phase as a taxonomy. SQL injection and XSS are the classes seen in practice so far.

On the job: This is shared vocabulary in code reviews and architecture conversations. When a developer asks why their code is flagged, you explain which OWASP category it falls into and why it matters.

Injection as a class

Any time user input reaches an interpreter without proper sanitization, injection is possible. SQL injection is one example.

On the job: When reviewing an API endpoint that accepts user input, a first question is whether the input reaches an interpreter, such as a database query. This question catches vulnerabilities automated tools miss.

The attacker’s mindset

The attacker’s mindset is the shift from checking that code works as intended to checking what it does with input it was not written to expect.

HTTP as the attack surface

Web vulnerabilities are delivered over HTTP: parameters, headers, cookies, request bodies, and additional parts of a request. Understanding HTTP underlies the web vulnerability classes in this plan.

On the job: Burp Suite sessions, manual web tests, and DAST scan results are built from HTTP transactions. Reading raw HTTP requests and responses is a routine part of the work.

II

Web & Network Foundations

Weeks 5–8

The major vulnerability classes live inside HTTP, authentication protocols, browser security models, or API design patterns. The phase reframes these layers from an adversarial perspective — not just how they work, but why they fail. Its audit of the Phase 0 FastAPI application produces write-ups labelled with the OWASP Top 10, OWASP's standard awareness document of the most critical security risks to web applications. It closes with the defender's view of AppSec (Alice and Bob) and cryptography fundamentals.

Learning Resources

- **"*Alice and Bob Learn Application Security*"** — Tanya Janca (SheHacksPurple), Wiley, 2020 — Covers threat modeling, security testing, and additional topics for securing modern software systems and architectures.
- **"*The Web Application Hacker's Handbook*"** — Dafydd Stuttard and Marcus Pinto, Wiley, 2011 — Foundational reference written by the creator of Burp Suite. Ch. 1–3 cover web application security, core defense mechanisms, and web application technologies. Ch. 6 covers authentication, Ch. 7 session management, Ch. 8 access controls. Some content is aging but these chapters remain the clearest treatment available.
- **PortSwigger Web Security Academy** — Free, online web security training from the creators of Burp Suite, with interactive labs.
- **MDN Web Docs** — An authoritative reference for HTTP, cookies, CORS, CSP, and the browser security model. Covers each security header, its syntax, and its security purpose in depth.
- **OWASP SSRF Prevention Cheat Sheet** — Covers the two SSRF cases, allowlist and resolved-address validation, IMDSv2 on AWS, deny-lists as a last resort, and additional guidance.
- **TryHackMe 'Pre-Security' path: Network Fundamentals module** — Primary resource for the networking week. Covers the Open Systems Interconnection (OSI) model, TCP/IP, ports, how packets travel, and additional topics in about two interactive hours. Not Net+, not CCNA — precisely scoped for AppSec-relevant networking.
- **TryHackMe 'Pre-Security' path: Putting It All Together room** — Covers how the components of a web request fit together: DNS, HTTP, load balancers, CDNs, databases, WAFs, and additional components, and how each layer relates to security decisions.
- **OWASP Cryptographic Storage Cheat Sheet** — Primary reference for the cryptography week. Covers which algorithms are acceptable and which are broken, key management, storage patterns, and additional guidance.
- **OWASP Application Security Verification Standard (ASVS) v5.0.0** — Approximately 350 testable security requirements covering authentication, access control, input validation, cryptography, API security, configuration, and additional areas. Level 1 is the first layer of defense; Level 2 covers standard security practice.
- **OWASP Top 10** — OWASP's standard awareness document of the most critical security risks to web applications.

- **TryHackMe: Writing Pentest Reports room** — The room that teaches the write-up structure this phase's write-ups use.
- **Burp Suite Documentation** — Official documentation for all Burp Suite tools: Proxy, Repeater, Intruder, Sequencer, Scanner, and Burp's browser.
- **FastAPI Official Documentation** — Official documentation for FastAPI, a web framework for building APIs with Python.
- **FastAPI Notes** — Your notes on the FastAPI documentation. They cover its tutorial and relevant advanced user guide sections.
- **OWASP Transport Layer Security Cheat Sheet** — TLS configuration, certificate management, and common misconfigurations. Companion to the crypto week.
- **hashcat** — Password cracking tool used in the cryptography week to crack an MD5 hash and make weak hashing concrete. The wiki's Dictionary attack and Example hashes pages give the exact commands for an MD5 wordlist attack.
- **freeCodeCamp: How to Crack Hashes with Hashcat** — An article on installing and working with Hashcat, a command line utility that cracks hashes.
- **bcrypt (Python library)** — Python library for bcrypt password hashing. Used in the cryptography week to implement correct password hashing in Python. Simple API: hash a password, verify a password.
- **OpenID Connect: How Connect Works** — The OpenID Foundation's overview of how OIDC adds an identity layer on top of OAuth 2.0.
- **OWASP SAML Security Cheat Sheet** — What breaks in SAML single sign-on (signature wrapping, replay, weak validation) and how to review it.

Web & Network Foundations — Weeks 5–8

01. READ: HTTP Deep Dive · Week 5

- Read *WAHH* Ch. 1–3 (web application security, core defense mechanisms, web application technologies) as foundational context
- Build on the Phase I Burp basics via the official documentation — Proxy, Repeater, and Intruder — the primary tools you will use throughout this phase
- Read CORS misconfigurations, CSP, TLS, HTTP methods, and status codes for their security implications
- **Full notes + note cards:** CORS misconfigurations, CSP, TLS, HTTP methods, and status codes with security implications

02. READ + LABS: Input Handling Fundamentals · Week 5

- Read the PortSwigger SQL injection learning path, then complete the following labs to internalize the input-validation root cause. Apprentice: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data and SQL injection vulnerability allowing login bypass
- **For each lab in this phase:** exploit it, then write lab notes for it if none exist
- **Full notes + note cards:** SQL injection root cause, parameterized queries, injection taxonomy

03. READ + LABS: Authentication & Session Management · Weeks 5–6

- Read *WAHH* Ch. 6 (Attacking Authentication) and Ch. 7 (Attacking Session Management)
- Complete the PortSwigger Authentication learning path, then the following labs:

  Apprentice:
  - Username enumeration via different responses
  - 2FA simple bypass
  - Password reset broken logic

  Practitioner:
  - Username enumeration via subtly different responses
  - Username enumeration via response timing
  - Broken brute-force protection, IP block
  - Username enumeration via account lock
  - 2FA broken logic
  - Brute-forcing a stay-logged-in cookie
  - Offline password cracking
  - Password reset poisoning via middleware
  - Password brute-force via password change
- Complete the JWT learning path, then the following labs:

  Apprentice:
  - JWT authentication bypass via unverified signature
  - JWT authentication bypass via flawed signature verification

  Practitioner:
  - JWT authentication bypass via weak signing key
  - JWT authentication bypass via jwk header injection
  - JWT authentication bypass via jku header injection

  Expert:
  - JWT authentication bypass via algorithm confusion
- Complete the OAuth learning path, then the following labs. Apprentice: Authentication bypass via OAuth implicit flow. Practitioner: Forced OAuth profile linking and OAuth account hijacking via redirect_uri
- Read how OpenID Connect layers identity on OAuth 2.0 and the OWASP SAML Security Cheat Sheet, and take notes on both. OAuth, OIDC, and SAML are the three protocols an authentication review meets
- **Full notes + note cards:** Password enumeration, session fixation, JWT algorithm confusion, OAuth redirect URI manipulation

04. READ + LABS: Access Control & APIs · Week 7

- Read *WAHH* Ch. 8 (Attacking Access Controls)
- Read the PortSwigger Access control learning path, then complete the following labs:

  Apprentice:
  - Unprotected admin functionality
  - Unprotected admin functionality with unpredictable URL
  - User role controlled by request parameter
  - User role can be modified in user profile
  - User ID controlled by request parameter

- User ID controlled by request parameter, with unpredictable user IDs
- User ID controlled by request parameter with data leakage in redirect
- User ID controlled by request parameter with password disclosure
- Insecure direct object references

• Read the PortSwigger API testing learning path, then complete the following lab. Apprentice: Exploiting an API endpoint using documentation

• **Full notes + note cards:** IDOR, horizontal/vertical privilege escalation, mass assignment, excessive data exposure

05. PROJECT: Authentication Audit · Week 7

• Read the OWASP ASVS authentication, authorization, and self-contained tokens chapters, and the Secret Management section of the configuration chapter

• Audit your Phase 0 FastAPI application against these chapters' requirements. Among other features, it implements OAuth2 with JWT and scoped authorization, with some security weaknesses left in deliberately, so the audit has findings to record

• Confirm each finding against the running application with Burp, opened in Burp's browser

• Record the findings in the write-up structure from Writing Pentest Reports

• Read the OWASP Top 10 pages for A01 Broken Access Control, A04 Cryptographic Failures, and A07 Authentication Failures, then label each write-up with its category

• Fix the findings in the application, with the Security and Settings & Env Variables sections of your FastAPI notes as references, plus add the fixed code to each write-up. Give remediation advice (no code) for brute-force protection

• Fix only these findings. The application's other deliberate weaknesses stay in place

06. STUDY: Networking Fundamentals for AppSec · Week 7

• Complete the TryHackMe 'Pre-Security' path — Network Fundamentals module. Take notes.

• Review existing notes for the URL-to-response flow.

• Read the OWASP SSRF Prevention Cheat Sheet, then the one-page 'DNS Rebinding' section in WAHH Ch. 13 for the mechanism the concept below relies on.

• Complete the TryHackMe 'Pre-Security' path — Putting It All Together room.

• **Full notes + note cards:** OSI layers 3/4/7 with one AppSec example each; TCP vs UDP with one security implication each; the URL request flow; common ports; private IP ranges; what a firewall, load balancer, and reverse proxy each do.

07. WRITE: Authentication Writeup + LinkedIn Post · Week 7

• Publish 800–1,200 word LinkedIn article: 'Five ways JWT authentication breaks'

• Lead with the JWT finding from the authentication audit of the Phase 0 FastAPI application. Then, cover the flaws from the following labs:

Apprentice:

- JWT authentication bypass via unverified signature
- JWT authentication bypass via flawed signature verification

Practitioner:

- JWT authentication bypass via weak signing key
- JWT authentication bypass via jwk header injection

• Write it from a developer's perspective. For each flaw, give the cause, the exploit, and the fix from How to prevent JWT attacks. For the audit finding, show the vulnerable code and the fixed code

08. READ: "*Alice and Bob Learn Application Security*" · Week 8

• Read "*Alice and Bob Learn Application Security*" Ch. 1–5 and Ch. 7: security fundamentals, security requirements, secure design, secure code, and common pitfalls (Ch. 1–5), and what an AppSec program does day-to-day (Ch. 7)

• Read during this week — this is the defender and program-building perspective that PortSwigger and *WAHH* do not cover

• **Key insight:** AppSec is not just finding vulnerabilities — it is building processes so developers stop introducing them

• **Full notes + note cards:** SDLC security touchpoints, developer security hygiene checklist, what an AppSec program actually does day-to-day

09. STUDY: Cryptography Fundamentals · Week 8

• Review existing sys design notes on auth and basic security before any new reading. The goal this week is the AppSec angle: how cryptographic primitives fail, what broken implementations look like in code, and how to flag them in a security review.

• Read the OWASP Cryptographic Storage Cheat Sheet and the OWASP Transport Layer Security Cheat Sheet. Take notes.

• **Hands-on:** Crack an MD5 hash with hashcat. Implement bcrypt password hashing in Python.

• **Full notes + note cards:** One card per algorithm type covered in the cheat sheets. Front: name and use case. Back: when to use it, what breaks it, what a broken implementation looks like in code.

Projects & Writings — Web & Network Foundations

Project: Authentication Audit

An audit of the Phase 0 FastAPI app against the OWASP ASVS authentication, authorization, and self-contained tokens chapters, and the Secret Management section of the configuration chapter. The findings are confirmed with Burp, recorded in the write-up structure from Writing Pentest Reports with the fixed code, and labelled with their OWASP Top 10 categories: A01 Broken Access Control, A04 Cryptographic Failures, and A07 Authentication Failures.

Writing: 'Five ways JWT authentication breaks' (800–1,200 words)

LinkedIn article leading with the audit finding, then four flaws from the PortSwigger JWT labs. Written from a developer's perspective: the cause, the exploit, and the fix.

Concepts and Their Use on the Job

HTTP security model

The HTTP headers that have security implications, what each does, and what happens when it is misconfigured.

On the job: When reviewing a PR that sets cookies or configures CORS, you check HttpOnly, Secure, SameSite, and Access-Control-Allow-Origin as a reflex.

Authentication and session security

How authentication protocols work and how they fail. JWT algorithm confusion is a recurring real-world finding.

On the job: JWT configuration is an early check: whether the algorithm is restricted to an allowlist, whether the signature is verified, and whether the claims are validated. These checks catch common JWT vulnerabilities.

Access control

IDOR, privilege escalation, horizontal vs. vertical access control flaws. Ranked A01 in the OWASP Top 10:2025.

On the job: When reviewing an API endpoint that returns data based on a user-supplied ID, you ask: is the user's authorization checked against the requested resource, or just their authentication? This question catches many IDOR flaws.

API security

REST API-specific vulnerability patterns — broken object-level authorization (IDOR), mass assignment, and excessive data exposure — and why APIs widen the attack surface beyond traditional server-rendered applications.

Input validation

The root cause of injection vulnerabilities. The injection classes share one root cause: unsanitized user input reaching an interpreter.

Cryptography fundamentals

Hashing vs. encryption vs. encoding. Which algorithms are broken (MD5, SHA-1 for passwords, DES, RC4) and which are correct (bcrypt, AES-GCM, RSA). How to identify weak crypto in a code review.

On the job: When you see a password hash, you check whether it is bcrypt/Argon2 (correct) or MD5/SHA-1 (flag immediately). When you see encrypted data, you check for AES-GCM (correct) vs AES-ECB (flag). When you see a secret, you check whether it is hardcoded or pulled from a secret manager.

Networking fundamentals for AppSec

OSI layers 3 (Network/IP), 4 (Transport/TCP-UDP), and 7 (Application/HTTP) and why AppSec lives at layer 7. TCP vs UDP and one security implication of each. The steps of a URL request from DNS query to HTTP response. Common ports and the services on them. Private IP ranges (10.x, 172.16–172.31.x, 192.168.x), localhost (127.0.0.1), and the AWS metadata endpoint (169.254.169.254). Standard web app network architecture: internet → firewall → load balancer → app server → database. What a firewall, load balancer, and reverse proxy each do and what security question to ask about each.

On the job: When asked 'where does TLS terminate?' in an architecture review, you answer and explain the security implication. When reviewing a deployment configuration, you check for unexpected open ports. When a threat model shows a service making outbound HTTP calls, you ask whether SSRF mitigations are in place and whether private IP ranges are blocked. When a design review describes traffic from an EC2 instance to 169.254.169.254 and asks if it is concerning, the answer is yes — SSRF against the metadata endpoint.

Networking as an attack surface

DNS rebinding exploits the gap between DNS validation and the TCP connection, bypassing SSRF protections. SSRF against private IP ranges and the AWS metadata endpoint (169.254.169.254) is a common high-severity finding in cloud-hosted applications. Where TLS terminates in an architecture tells you whether internal traffic is encrypted; HTTP statelessness is why cookies exist and therefore an attack surface; and unexpected open ports in a deployment configuration are a finding. These networking behaviors are what AppSec engineers encounter in real work, not networking-engineering concepts.

AppSec program thinking (Janca)

AppSec is not just finding bugs — it is building the processes, training, and SDLC integration that stop bugs from being introduced. The developer’s perspective on the defensive side of the role.

III

Core Vulnerability Classes

Weeks 9–15

Phase III is the technical core of the plan. This phase goes deep on each vulnerability class it names — not survey knowledge, but genuine understanding of root cause, exploitation mechanics, and remediation. Its write-ups are labelled with their OWASP Top 10 categories: lab write-ups for A01 Broken Access Control, A05 Injection, A08 Software or Data Integrity Failures, and A10 Mishandling of Exceptional Conditions, and a write-up of an A06 Insecure Design flaw in the Phase 0 FastAPI application. Complete the labs each step names. Four classes are marked optional, exam route: the Burp Suite Certified Practitioner exam covers all four, so they return with it.

Learning Resources

- **PortSwigger Web Security Academy** — Free, online web security training from the creators of Burp Suite, with interactive labs.
- **OWASP Top 10** — OWASP's standard awareness document of the most critical security risks to web applications.
- **OWASP API Security Top 10** — The API-specific companion to the web Top 10. Covers BOLA (Broken Object Level Authorization), broken authentication, BOPLA (Broken Object Property Level Authorization, which merges the older mass assignment and excessive data exposure categories), BFLA (Broken Function Level Authorization), SSRF, and additional categories as they manifest in REST APIs.
- **"*Alice and Bob Learn Application Security*"** — Tanya Janca (SheHacksPurple), Wiley, 2020 — Covers threat modeling, security testing, and additional topics for securing modern software systems and architectures.
- **"*Hacking APIs*"** — Corey Ball, No Starch Press, 2022 — REST API security testing. Ch. 1–3 cover how web applications and APIs work and the common API vulnerability classes. Ch. 6–9 cover discovery, endpoint analysis, authentication attacks, and fuzzing.
- **OWASP Top 10 for LLM Applications** — OWASP's risk list for LLM-backed apps: prompt injection, improper output handling, excessive agency, and sensitive information disclosure.
- **OWASP SSRF Prevention Cheat Sheet** — Covers the two SSRF cases, allowlist and resolved-address validation, IMDSv2 on AWS, deny-lists as a last resort, and additional guidance.
- **Turbo Intruder** — Turbo Intruder is a Burp Suite extension for sending large numbers of HTTP requests and analyzing the results. You can install BApps directly within Burp, via the BApp Store feature in the Burp Extender tool.
- **React documentation: Dangerously setting the inner HTML** — The section of the React documentation on passing a raw HTML string to an element, and the XSS risk it carries.
- **TryHackMe: Writing Pentest Reports room** — The room that teaches the write-up structure this phase's write-ups use.

Core Vulnerability Classes — Weeks 9–15

01. READ + LABS: SQL Injection: Weeks 9–10

- **Buy Burp Suite Professional now** (~$499/yr) — the out-of-band labs in this phase need Burp Collaborator, which is Pro-only. Buying at the start of Phase III lets you do every lab here in place instead of skipping the Collaborator-based ones, and you use Pro continuously from here through the BSCP exam. See Burp Suite Professional.
- Re-read your Phase II SQL injection notes (the learning path stays linked if you need it), then complete the following labs (the out-of-band labs use Burp Collaborator, and you now have Pro):

  Practitioner:
  - SQL injection attack, querying the database type and version on Oracle
  - SQL injection attack, querying the database type and version on MySQL and Microsoft
  - SQL injection UNION attack, determining the number of columns returned by the query
  - SQL injection UNION attack, finding a column containing text
  - SQL injection UNION attack, retrieving data from other tables
  - SQL injection UNION attack, retrieving multiple values in a single column
  - Blind SQL injection with conditional responses
  - Blind SQL injection with conditional errors
  - Blind SQL injection with time delays
  - Blind SQL injection with time delays and information retrieval
  - Blind SQL injection with out-of-band interaction
  - Blind SQL injection with out-of-band data exfiltration
- Read the OWASP Top 10 page for A05 Injection. Then, write up Blind SQL injection with conditional responses in the write-up structure from Writing Pentest Reports, and label it A05 Injection
- **For each lab in this phase:** exploit it, then write lab notes for it if none exist

02. READ + LABS: OS Command Injection: Week 10

- Work through the PortSwigger OS command injection learning path, then the following labs:

  Apprentice:
  - OS command injection, simple case

  Practitioner:
  - Blind OS command injection with time delays
  - Blind OS command injection with output redirection
  - Blind OS command injection with out-of-band data exfiltration

03. OPTIONAL, EXAM ROUTE. READ + LABS: Server-Side Template Injection: Week 10

- Take this step only if you sit the Burp Suite Certified Practitioner exam, which covers this class
- Work through the PortSwigger Server-side template injection learning path, then the following labs. Practitioner: Basic server-side template injection and Basic server-side template injection (code context)

04. READ + LABS: Information Disclosure: Week 10

- Read the PortSwigger Information disclosure learning path, then complete the following lab. Apprentice: Information disclosure in error messages
- Read the OWASP Top 10 page for A10 Mishandling of Exceptional Conditions. Then, write up Information disclosure in error messages in the write-up structure from Writing Pentest Reports, and label it A10 Mishandling of Exceptional Conditions

05. READ + LABS: Cross-Site Scripting: Week 11

- Complete the PortSwigger Cross-site scripting learning path, then the following labs:

  Apprentice:
  - Reflected XSS into HTML context with nothing encoded
  - Stored XSS into HTML context with nothing encoded
  - DOM XSS in document.write sink using source location.search
  - DOM XSS in innerHTML sink using source location.search
  - DOM XSS in jQuery anchor href attribute sink using location.search source
  - DOM XSS in jQuery selector sink using a hashchange event
  - Reflected XSS into attribute with angle brackets HTML-encoded
  - Stored XSS into anchor href attribute with double quotes HTML-encoded
  - Reflected XSS into a JavaScript string with angle brackets HTML encoded

  Practitioner:
  - Reflected DOM XSS
  - Stored DOM XSS
  - Exploiting cross-site scripting to steal cookies
  - Exploiting cross-site scripting to capture passwords
  - Exploiting XSS to bypass CSRF defenses
- Read Dangerously setting the inner HTML in the React documentation

06. READ + LABS: CSRF: Week 11

- Complete the PortSwigger CSRF learning path, then the following labs (each is a distinct broken-defense pattern):

  Apprentice:
  - CSRF vulnerability with no defenses

  Practitioner:
  - CSRF where token validation depends on request method
  - CSRF where token validation depends on token being present
  - CSRF where token is not tied to user session
  - CSRF where token is tied to non-session cookie
  - CSRF where token is duplicated in cookie
  - SameSite Lax bypass via method override
  - SameSite Strict bypass via client-side redirect
  - SameSite Strict bypass via sibling domain
  - CSRF where Referer validation depends on header being present
  - CSRF with broken Referer validation

07. READ + LABS: SSRF: Week 11

- Complete the PortSwigger SSRF learning path, then the following labs:

  Apprentice:
  - Basic SSRF against the local server
  - Basic SSRF against another back-end system

  Practitioner:
  - Blind SSRF with out-of-band detection
  - SSRF with blacklist-based input filter
  - SSRF with filter bypass via open redirection vulnerability

- Write up SSRF with blacklist-based input filter in the write-up structure from Writing Pentest Reports, with remediation advice from the OWASP SSRF Prevention Cheat Sheet, and label it with the OWASP Top 10 category whose page lists Server-Side Request Forgery among its mapped CWEs

08. OPTIONAL, EXAM ROUTE. READ + LABS: XXE: Week 11

- Take this step only if you sit the Burp Suite Certified Practitioner exam, which covers this class
- Complete the PortSwigger XML external entity (XXE) learning path, then the following labs. Apprentice: Exploiting XXE using external entities to retrieve files and Exploiting XXE to perform SSRF attacks. Practitioner: Exploiting blind XXE to retrieve data via error messages

09. READ + LABS: Access Control: Week 12

- Review your Phase II Access control notes, then complete the following labs:

  Practitioner:
  - URL-based access control can be circumvented
  - Method-based access control can be circumvented
  - Multi-step process with no access control on one step
  - Referer-based access control

10. READ + LABS: Business Logic Vulnerabilities: Week 12

- Work through the PortSwigger Business logic vulnerabilities learning path, then the following labs (each is a distinct logic flaw):

  Apprentice:
  - Excessive trust in client-side controls
  - High-level logic vulnerability
  - Inconsistent security controls
  - Flawed enforcement of business rules

  Practitioner:
  - Low-level logic flaw
  - Inconsistent handling of exceptional input
  - Weak isolation on dual-use endpoint
  - Insufficient workflow validation
  - Authentication bypass via flawed state machine
  - Infinite money logic flaw
  - Authentication bypass via encryption oracle

- Read the OWASP Top 10 page for A06 Insecure Design

- Confirm with Burp that the purchase route of the Phase 0 FastAPI application accepts the manager discount flag from the request body, the flaw from Excessive trust in client-side controls
- Write up the finding in the write-up structure from Writing Pentest Reports, and label it A06 Insecure Design
- Fix the finding in the application, plus add the fixed code to the write-up
- Fix only this finding. The application's other deliberate weaknesses stay in place

11. **READ + LABS: API Security Deep Dive:** Weeks 12–15
    - Read the 2023 edition of the OWASP API Security Top 10, then the 2023 risk pages that its table links to
    - Read "*Hacking APIs*" Ch. 1–3 (how web applications and APIs work, and the common API vulnerability classes) and Ch. 6–9 (API discovery and reconnaissance, endpoint analysis, authentication attacks, and fuzzing)
    - Read "*Alice and Bob Learn Application Security*" Ch. 6 (testing and deployment) and Ch. 8 (securing modern applications and systems — APIs, microservices, serverless)
    - Review your Phase II API testing notes, then complete the following labs:

      Practitioner:
      - Exploiting server-side parameter pollution in a query string
      - Finding and exploiting an unused API endpoint
      - Exploiting a mass assignment vulnerability

      Expert:
      - Exploiting server-side parameter pollution in a REST URL

    - For each lab, write down the API design decision that created the vulnerability
12. **READ + LABS: Insecure Deserialization:** Week 13
    - Complete the PortSwigger Insecure deserialization learning path, then the following labs:

      Apprentice:
      - Modifying serialized objects

      Practitioner:
      - Modifying serialized data types
      - Using application functionality to exploit insecure deserialization
      - Exploiting Java deserialization with Apache Commons

    - Read the OWASP Top 10 page for A08 Software or Data Integrity Failures. Then, write up Modifying serialized objects in the write-up structure from Writing Pentest Reports, and label it A08 Software or Data Integrity Failures
13. **OPTIONAL, EXAM ROUTE. READ + LABS: File Upload:** Week 13
    - Take this step only if you sit the Burp Suite Certified Practitioner exam, which covers this class
    - Complete the PortSwigger File upload learning path, then the following labs:

      Apprentice:
      - Remote code execution via web shell upload
      - Web shell upload via Content-Type restriction bypass

      Practitioner:
      - Web shell upload via path traversal

- Web shell upload via extension blacklist bypass
- Web shell upload via obfuscated file extension
- Remote code execution via polyglot web shell upload

Expert:

- Web shell upload via race condition (the lab's solution uses the Turbo Intruder extension)

14. OPTIONAL, EXAM ROUTE. READ + LABS: Path Traversal: Week 13

- Take this step only if you sit the Burp Suite Certified Practitioner exam, which covers this class
- Complete the PortSwigger Path traversal learning path, then the following labs (each is a distinct filter bypass):

Apprentice:

- File path traversal, simple case

Practitioner:

- File path traversal, traversal sequences blocked with absolute path bypass
- File path traversal, traversal sequences stripped non-recursively
- File path traversal, traversal sequences stripped with superfluous URL-decode
- File path traversal, validation of start of path
- File path traversal, validation of file extension with null byte bypass

15. READ + LABS: Web LLM Attacks (PortSwigger): Week 15

- Complete the Web LLM attacks learning path, then the following labs:

Apprentice:

- Exploiting LLM APIs with excessive agency

Practitioner:

- Exploiting vulnerabilities in LLM APIs
- Indirect prompt injection

Expert:

- Exploiting insecure output handling in LLMs

16. READ: OWASP Top 10 for LLM Applications: Week 15

- Read the entries of the OWASP Top 10 for LLM Applications
- Map each Web LLM attacks lab you solved to its entry in the OWASP Top 10 for LLM Applications

Projects & Writings — Core Vulnerability Classes

Writing: Lab write-ups

Write-ups of the following labs in the write-up structure from Writing Pentest Reports: SSRF with blacklist-based input filter, labelled A01 Broken Access Control; Blind SQL injection with conditional responses, labelled A05 Injection; Information disclosure in error messages, labelled A10 Mishandling of Exceptional Conditions; and Modifying serialized objects, labelled A08 Software or Data Integrity Failures.

Writing: Insecure Design write-up

A write-up of the manager discount flaw in the Phase 0 FastAPI application: confirmed with Burp, written in the write-up structure from Writing Pentest Reports with the fixed code, and labelled A06 Insecure Design.

Concepts and Their Use on the Job

Injection — complete taxonomy

SQL, command, template, LDAP, NoSQL. Root cause: unsanitized user input reaching an interpreter. Fix: parameterized queries.

On the job: a database query, a shell command or a template render gets the same question: does user input reach this without sanitization?

XSS — all three types

Reflected, stored, DOM. Root cause: user input rendered as HTML without encoding. Fix: context-appropriate output encoding.

On the job: React escapes JSX output by default, which prevents many reflected XSS vectors. What to check in a React code review: dangerouslySetInnerHTML usage (bypasses JSX escaping entirely), third-party component XSS vectors, and any server-rendered template output that feeds into the React tree.

CSRF and SSRF

CSRF exploits browser trust. SSRF exploits server trust to reach internal resources. Fix: SameSite cookies for CSRF; URL allowlists for SSRF.

On the job: An endpoint accepting a URL parameter gets flagged for review. Does it fetch that URL server-side? Is the response returned to the user? Is there a URL allowlist? Three questions.

Access control flaws

Ranked A01 in the OWASP Top 10:2025. Fix: verify the authenticated user is authorized for the specific resource on every request.

Business logic vulnerabilities

Hard for automated tools to detect. Requires understanding intended behavior.

On the job: This is the vulnerability class that threat modeling catches before it ships. STRIDE analysis of a data flow diagram surfaces the business logic assumptions that could be violated.

Securing LLM-backed features

The LLM-specific classes — prompt injection, improper output handling, excessive agency, and sensitive information disclosure — sit on top of the classic vulnerability classes covered in this phase: injection is injection, output handling is untrusted-input handling, excessive agency is least privilege.

On the job: An increasingly common AppSec task; review an LLM-backed endpoint with the same questions you ask of any endpoint that takes untrusted input and calls tools.

IV

Security Testing & Tooling

Weeks 16–22

Phase IV moves from knowing vulnerability classes to operating the professional toolchain used to find them systematically. Burp Suite is used throughout the phase. SAST (static analysis of source code, via Semgrep) and DAST (dynamic testing of the running app) are the automation layer. The phase builds tool fluency: Burp Suite finds vulnerabilities faster once the classes from Phase III are understood. Its dependency scanning step produces a write-up of the Phase 0 FastAPI application's supply chain, labelled with its OWASP Top 10 category, A03 Software Supply Chain Failures. The phase also covers the BSCP exam topics that earlier phases did not, including HTTP request smuggling, Host header attacks, and web cache poisoning.

Learning Resources

- **Burp Suite Documentation** — Official documentation for all Burp Suite tools: Proxy, Repeater, Intruder, Sequencer, Scanner, and Burp's browser.
- **Semgrep Documentation** — Primary for general SAST. An open-source SAST tool.
- **Learn Semgrep Syntax** — Semgrep's interactive tutorials on writing rules: Basics, Pattern composition, and Advanced features.
- **Bandit Documentation** — Python-specific SAST tool. Detects insecure subprocess calls, eval() usage, weak SSL contexts, hardcoded passwords, insecure pickle and yaml usage.
- **GitHub code scanning (CodeQL)** — GitHub's built-in SAST, powered by CodeQL, enabled per repository with a default setup. Used as the second SAST engine next to Semgrep. CodeQL query docs.
- **DVWA (Damn Vulnerable Web Application)** — Deliberately insecure PHP/MariaDB practice application, run locally via Docker Compose (localhost:4280). The second DAST target alongside WebGoat; set up in step 04 and scanned with ZAP in step 05.
- **ZAP Documentation** — A free, open-source DAST tool (formerly an OWASP project): an intercepting proxy plus an automated scanner that crawls a running app and runs active and passive checks for common web vulnerabilities. A free alternative to Burp Scanner, with a baseline scan that wires easily into CI.
- **Snyk Documentation** — Software composition analysis (SCA) — finding vulnerabilities in third-party dependencies. Available with a free tier.
- **Syft and CycloneDX** — Syft generates a software bill of materials (SBOM) from an image or source tree; CycloneDX is the SBOM standard it emits. Used in the Snyk step. (CycloneDX)
- **Grype** — A vulnerability scanner for container images, filesystems, and SBOMs. Used in the Snyk step to scan the Syft SBOM.
- **OWASP Top 10** — OWASP's standard awareness document of the most critical security risks to web applications.
- **TryHackMe: Writing Pentest Reports room** — The room that teaches the write-up structure this phase's write-ups use.
- **PortSwigger Web Security Academy** — Free, online web security training from the creators of Burp Suite, with interactive labs.

- **FastAPI Official Documentation** — Official documentation for FastAPI, a web framework for building APIs with Python.
- **Phase 0 FastAPI app** — Your own repository — the SAST scan target for Semgrep's Python ruleset and Bandit (Bandit is Python-only).
- **OWASP WebGoat** — A deliberately insecure web application, maintained by OWASP, designed to teach web application security lessons.
- **OWASP Juice Shop** — Deliberately insecure modern web application. The primary target for this phase's full security assessment: Burp Suite manual testing, ZAP DAST scan, Semgrep SAST, Snyk SCA. Runs via Docker or Node.
- **Pwning OWASP Juice Shop** — The official companion guide. Groups every Juice Shop challenge by vulnerability class and has full solutions in its appendix; the structure for the Phase IV full assessment.
- **TruffleHog** — Secrets scanner that searches full git history for leaked credentials, including credentials committed and later deleted.
- **Gitleaks** — Lightweight secrets scanner. Runs as a pre-commit hook and CI gate. Scans the repo history by default; as a pre-commit hook it scans only the staged diff, fast enough for every PR.
- **pre-commit** — A framework for managing git pre-commit hooks. Its opening explains what a pre-commit hook is; Gitleaks runs as one.
- **GitHub Actions quickstart** — GitHub's guide to creating a first workflow file in a repository.
- **GitHub documentation: Removing sensitive data from a repository** — What to do when a secret has been committed: revoke or rotate it first, then decide whether to rewrite history.
- **OWASP Threat Modeling Cheat Sheet** — A concise, actionable reference for those new to threat modeling and those seeking a refresher. It uses STRIDE for illustration.
- **OWASP Top 10 for LLM Applications** — OWASP's risk list for LLM-backed apps: prompt injection, improper output handling, excessive agency, and sensitive information disclosure.

Security Testing & Tooling — Weeks 16–22

01. LEARN + LABS: Burp Suite Mastery: Week 16

- Proxy, Repeater, and Intruder are already yours from Phase II — review those notes; no need to re-read the docs
- Two more Community-edition tools: Decoder (encode/decode/hash inline) and Comparer (visual diff of two messages)
- Practice Decoder and Comparer on the following labs. Apprentice: Username enumeration via different responses. Practitioner: SQL injection with filter bypass via XML encoding and Blind SQL injection with conditional responses
- **By end of Week 16:** fluent with Decoder and Comparer, and comfortable with Proxy match-and-replace and Repeater tab groups

02. LEARN + LABS: Semgrep & Bandit — SAST Tooling: Week 16

- Install the Semgrep CLI, then run semgrep scan with its default Registry rules and Bandit against your own Phase 0 FastAPI app
- Triage the results: distinguish true positives from false positives
- Complete the tutorials at Learn Semgrep Syntax
- Write a custom Semgrep rule that catches a SQL query built by joining strings, then run it as a local rule file against WebGoat's SqlInjectionLesson5a.java
- Write a second custom Semgrep rule that catches user input joined into an os.system call, and run it against the app.py file from the quickstart
- Complete the GitHub Actions quickstart
- Integrate Semgrep into a GitHub Actions workflow
- Enable GitHub code scanning (CodeQL default setup) on the FastAPI repo, let it run once, and read the results next to Semgrep's: same codebase, two engines, note what each catches that the other does not.
- Bandit: install bandit with pip. Run against a sample Python project and against the Phase 0 FastAPI project. Look for: B101 (assert usage), B102 (exec usage), B105/B106/B107 (hardcoded password), B201 (Flask debug mode), B301 (pickle), B501–B504 (SSL/TLS issues). The finding IDs become vocabulary you use in code review conversations
- Run both tools against the same Python codebase and compare results. Bandit catches Python-specific antipatterns that Semgrep misses without custom rules. Semgrep catches logic-level patterns that Bandit's fixed ruleset cannot express. Together with the Gitleaks and Snyk gates this is a DevSecOps pipeline: every pull request is scanned, and a failing security check blocks the merge.

03. LEARN: Secrets Scanning — TruffleHog & Gitleaks: Week 16

- Install TruffleHog, then run its README example Scan a repo for only verified secrets
- Run TruffleHog against the local git repository of the Phase 0 FastAPI application by following Scan a local git repo
- Read the opening of pre-commit, which explains what a pre-commit hook is. Then, install Gitleaks and set it up as a pre-commit hook in the repository of the Phase 0 FastAPI application
- Add the Gitleaks action and the TruffleHog action to a GitHub Actions workflow in that repository
- Read Removing sensitive data from a repository in the GitHub documentation. Then, rotate the JWT signing key of the Phase 0 FastAPI application, which sits in its git history, with the command from Handle JWT tokens
- **Full notes + note cards:** When to use TruffleHog vs Gitleaks, what a false positive looks like, what to do when a live secret is found (rotation procedure, not just deletion)

04. SETUP: DVWA — Second DAST Target: Week 16

- **DVWA** (Damn Vulnerable Web Application) is a second deliberately insecure practice app, written in PHP — same purpose as WebGoat, different codebase. The next step scans both targets
- Install via Docker Compose from the official repository: clone digininja/DVWA and run **docker compose up -d**. It serves on http://localhost:4280 (not port 80).

- First run: log in with the default credentials (admin / password), open **Setup DVWA** from the menu and click **Create / Reset Database**. Set the security level to **Low** under DVWA Security so the scanner in the next step has findings to report
- Confirm both practice apps are reachable side by side: WebGoat (from Phase 0) and DVWA. The troubleshooting section covers the common start-up errors

05. LEARN: ZAP — DAST Basics: Week 16

- Read the ZAP Getting Started guide, then install ZAP
- Run the guide's automated scan against WebGoat and against DVWA
- WebGoat and DVWA both sit behind a login, which the automated scan cannot pass. Log in to each through a browser launched from ZAP and explore it manually, so ZAP records alerts for the pages behind the login
- Compare the alerts ZAP raised for the two applications
- Triage the alerts: true positive, false positive, needs investigation

06. OPTIONAL, EXAM ROUTE. DOCUMENT: Save Exam-Ready Proof-of-Concept Exploits: Throughout Weeks 16–20

- Take this step only if you sit the Burp Suite Certified Practitioner exam
- PortSwigger publishes full proof-of-concept (PoC) exploit and solution code for every lab. As you clear each practitioner lab this phase, write lab notes for it if none exist, and check that the notes hold the working exploit. The BSCP is open to your own notes (see PortSwigger's prep guidance).
- Digital notes are ideal — the exam is fully open-book (your own notes, the web, third-party tools, and Burp extensions are all allowed) and you sit it in your own browser alongside Burp, so keep everything in one searchable file (Markdown or plain text) or a notes app rather than on paper.
- Keep payloads ready to use, and pre-load your payload lists and useful extensions into Burp before you start.
- Pre-adapt the high-value PoCs for exam conditions: turn XSS print()/alert() payloads into cookie-stealers, parameterize your SQLi and SSRF payloads, and keep Collaborator / OOB templates ready to paste.
- Organize them by vulnerability class so they are quick to find during the exam

07. LABS: Out-of-Band (OOB) Exploitation with Burp Collaborator: Week 17

- **Requires Burp Suite Professional** — Burp Collaborator is Pro-only, and nearly every blind / out-of-band variant on the BSCP routes through it. You first used Collaborator in Phase III (Pro from the start); this step is a focused OOB drill to lock in exam fluency — re-confirm each one fast.
- **Blind SQL injection:** re-read your notes on blind SQL injection (the learning path stays linked), then the following labs. Practitioner: Blind SQL injection with out-of-band interaction and Blind SQL injection with out-of-band data exfiltration
- **Blind XXE:** re-read your notes on blind XXE (the learning path stays linked), then the following labs. Practitioner: Blind XXE with out-of-band interaction and Exploiting blind XXE to exfiltrate data using a malicious external DTD
- **Blind OS command injection:** from the OS command injection learning path, the following labs. Practitioner: Blind OS command injection with out-of-band interaction and Blind OS command injection with out-of-band data exfiltration

- Carry the same Collaborator technique into blind SSTI detection and any asynchronous sink — fire a DNS/HTTP callback to confirm the bug, then exfiltrate data through it
- **For each lab:** exploit it, then write lab notes for it if none exist, and check that the notes hold the Collaborator payload

08. LEARN: Snyk — Dependency Scanning: Week 17

- Install and authenticate the Snyk CLI, then follow CLI support for Python to run *snyk test* against the requirements.txt file of the Phase 0 FastAPI application
- Generate a CycloneDX SBOM of the application with Syft, then scan the SBOM with Grype
- Read the OWASP Top 10 page for A03 Software Supply Chain Failures. Then, audit how the application tracks, pins, and scans its dependencies against the "You are likely vulnerable if" list on that page
- Write up the findings, with the Snyk and Grype results as evidence, in the write-up structure from Writing Pentest Reports, and label them A03 Software Supply Chain Failures
- Wire *snyk test* into a GitHub Actions workflow with the Snyk Python action, alongside Semgrep and Gitleaks, so newly disclosed dependency CVEs fail the build
- Add the fixed code to the write-up. Give remediation advice (no code) for pinning nested dependencies
- Fix only these findings. The application's other deliberate weaknesses stay in place

09. PROJECT: Secure a Small LLM-Backed API: Week 17

- Add one small LLM-backed endpoint to your Phase 0 FastAPI app — a /summarize or support-reply route that calls a hosted model API (for example the Anthropic or OpenAI API).
- Before attacking it, sketch a one-page STRIDE pass on the new endpoint: the trust boundaries between user, model, and any tool the model can call. A warm-up for the full threat model in Phase VI
- Run the same three attack classes you practised in your Phase III Web LLM Attacks step against your own endpoint: prompt injection (LLM01); improper output handling (LLM05) — show the XSS or SQL injection that trusting model output enables, then fix it with encoding and parameterization; and excessive agency (LLM06) — least privilege plus an approval step for any tool the model can call
- Write it up in your standard report format and push it to GitHub; label each finding with its OWASP LLM Top 10 ID (LLM01, LLM05, and so on).
- Scope: this is securing an LLM *application*, not machine learning — stay on the AppSec slice

10. READ + LABS: Remaining BSCP Topics: Weeks 17–18

- Read the HTTP request smuggling learning path, then complete the following labs:

  Practitioner:
  - HTTP request smuggling, basic CL.TE vulnerability
  - HTTP request smuggling, basic TE.CL vulnerability
  - HTTP request smuggling, obfuscating the TE header
  - Exploiting HTTP request smuggling to capture other users' requests (a BSCP favorite)
- Read the HTTP Host header attacks learning path, then complete the following labs:

  Apprentice:
  - Basic password reset poisoning
  - Host header authentication bypass

Practitioner:

- Web cache poisoning via ambiguous requests
- Routing-based SSRF

- Read the Web cache poisoning learning path, then complete the following labs:

Practitioner:

- Web cache poisoning with an unkeyed header
- Web cache poisoning with an unkeyed cookie
- Web cache poisoning with multiple headers
- Targeted web cache poisoning using an unknown header

11. **LEARN + LABS: Targeted Scanning & Content Discovery:** Week 18

- **Requires Burp Suite Professional** (the Scanner and Engagement tools are Pro-only). The exam is time-boxed, so you scan smart, not broad — this is an explicit BSCP skill.
- Read using Burp Scanner during manual testing, then the following labs. Practitioner: Discovering vulnerabilities quickly with targeted scanning and Scanning non-standard data structures. Right-click a single request → *Do active scan* to audit just that request in seconds instead of crawling the whole site.
- **Content discovery:** use Burp's Engagement tools → *Discover content* to enumerate hidden directories, files, and parameters (for example a hidden /admin) — a recurring first move on exam apps. Practice it on labs you have already solved.
- **Build the habit:** hand every new request to the Scanner while you test manually, and run content discovery before assuming an app's surface is fully mapped

12. **OPTIONAL, EXAM ROUTE. DRILL: PortSwigger's Recommended Reinforcement Labs:** Week 18

- Take this step only if you sit the Burp Suite Certified Practitioner exam
- Re-solve cold the following labs from earlier steps, which PortSwigger recommends before the exam:

Practitioner:

- Exploiting cross-site scripting to steal cookies
- Blind SQL injection with out-of-band data exfiltration
- Forced OAuth profile linking
- Brute-forcing a stay-logged-in cookie
- Exploiting HTTP request smuggling to capture other users' requests
- SSRF with blacklist-based input filter
- SQL injection with filter bypass via XML encoding
- Discovering vulnerabilities quickly with targeted scanning

- Check that the lab notes for each lab hold the working exploit

13. **OPTIONAL, EXAM ROUTE. PROJECT: Burp Suite Certified Practitioner Prep:** Weeks 18–19

- Take this step only if you sit the Burp Suite Certified Practitioner exam
- Requires Burp Suite Professional (bought at the start of Phase III); the BSCP exam itself is ~$99 per attempt
- Pass the BSCP practice exam in under 2 hours — and do it more than once — before booking the real exam. It is one application in two hours, half the real exam, so consistently passing it under time is the strongest signal you are ready

14. **OPTIONAL, EXAM ROUTE. DRILL: Fluency — Re-Solve Cold:** Week 19
    - Take this step only if you sit the Burp Suite Certified Practitioner exam
    - For each major class, take one practitioner lab you have already solved and re-solve it cold — no notes, no solution — in 15 minutes or less.
    - If a lab runs past 15 minutes or you reach for your notes, that class is not yet exam-fluent — drill it again.
    - This is the gap between having seen a vulnerability and being able to exploit it under time pressure with no hints — which is what the exam tests
15. **OPTIONAL, EXAM ROUTE. DRILL: Mystery Labs — Cold Recognition:** Week 19
    - Take this step only if you sit the Burp Suite Certified Practitioner exam
    - The exam gives you no vulnerability hints — you must recognize the class from behavior alone. The mystery lab challenge starts a random lab with its title and description hidden, the closest practice for that.
    - Solve at least 5 mystery labs cold — no hints, working only from recon — before booking the exam. Time each one; aim for fast, confident classification.
    - For a class that needs more work, go back to that topic's labs and notes, then return to the mystery challenge
16. **OPTIONAL, EXAM ROUTE. DRILL: Exam Stamina — Six Labs in One Block:** Week 19
    - Take this step only if you sit the Burp Suite Certified Practitioner exam
    - The real exam is a single continuous 4-hour session — two applications, three stages each, and a pass needs every stage. Rehearse that endurance before you book it.
    - Take six practitioner labs you have not memorized and solve them back-to-back in one timed 4-hour block — own notes only, Burp Pro, Collaborator live, no long breaks.
    - Track time per lab and practice triage: if one runs past budget, bank partial progress, move on, and circle back — the time management is as much the test as the exploitation
17. **PROJECT: Full Security Assessment of Juice Shop:** Weeks 19–21
    - Conduct a full security assessment of OWASP Juice Shop using: Burp Suite (manual), ZAP (DAST), Semgrep with its JavaScript/TypeScript rulesets (SAST), Snyk (dependencies), TruffleHog/Gitleaks (secrets)
    - Work through it systematically using the official companion guide Pwning OWASP Juice Shop, which groups every challenge by vulnerability class — injection, broken authentication, XSS, broken access control, and more — so you can map each finding to an OWASP category
    - Reuse the systematic methodology and the write-up structure from Writing Pentest Reports — this project is where the full workflow comes together: recon, tooling, manual exploitation, false-positive triage, and professional reporting
    - Produce a written vulnerability report: executive summary, findings table (vuln, severity, CVSS, evidence, recommendation), detailed finding writeups
    - Export the finished report as a standalone PDF
    - It demonstrates the AppSec workflow from tool setup to professional reporting

18. OPTIONAL, EXAM ROUTE. EXAM: Burp Suite Certified Practitioner: Weeks 20–22

- Take this step only if you sit the Burp Suite Certified Practitioner exam
- Take the BSCP exam. The exam consists of two applications, each completed in three stages: access a user account, reach the admin interface, read /home/carlos/secret. 4 hours.
- PortSwigger sets no limit on attempts (each attempt costs $99).

Projects & Writings — Security Testing & Tooling

Project: Full Security Assessment of OWASP Juice Shop

Burp manual testing, ZAP, Semgrep, Snyk, and secrets scanning against one target, written up as a professional vulnerability report (executive summary, findings table, detailed findings) and exported as a standalone PDF.

Project: Secure a Small LLM-Backed API

An LLM-backed endpoint added to the Phase 0 FastAPI app, attacked for prompt injection, improper output handling, and excessive agency, then fixed; writeup labelled with OWASP LLM Top 10 IDs.

Project: Custom Semgrep rules and CI security workflow

A custom Semgrep rule that catches a SQL query built by joining strings, and a second rule that catches user input joined into an os.system call, plus a GitHub Actions workflow running Semgrep, Gitleaks, TruffleHog, and Snyk.

Writing: Dependency scanning write-up

A write-up of how the Phase 0 FastAPI application tracks, pins, and scans its dependencies, with the Snyk and Grype results as evidence and the fixed code, labelled A03 Software Supply Chain Failures.

Certification: Burp Suite Certified Practitioner

Passed exam; a credential that shows hands-on web-exploitation skill.

Concepts and Their Use on the Job

Burp Suite as an extension of manual thinking

Burp Suite intercepts, replays, and scans HTTP traffic, among other functions. Its manual tools do not find vulnerabilities on their own — they make manual exploitation efficient. Understanding of what to look for drives the tool, not the reverse.

On the job: Web pentest engagements commonly run through Burp Suite or an equivalent proxy. It shows what a web application sends and receives. The AppSec engineer’s equivalent of a developer’s debugger.

SAST vs. DAST vs. SCA vs. Secrets Scanning

Four distinct scanning approaches that catch different vulnerability classes. SAST reads code, misses runtime behavior. DAST tests a running app, misses code paths not exercised. SCA finds known CVEs in dependencies. Secrets scanning finds exposed credentials in code and history. A mature AppSec program typically uses all four — plus Python-specific tools like Bandit for Python shops.

On the job: Integrating Semgrep into a company’s CI/CD pipeline, writing rules for company-specific vulnerability patterns, and triaging results is a common AppSec responsibility. The custom rule you built is the portfolio demonstration.

False positive triage

Automated tools have noise. Distinguishing real findings from false positives without dismissing everything is where security judgment matters more than tool knowledge.

Professional vulnerability reporting

Severity rating (CVSS), evidence documentation, clear remediation guidance. The output of AppSec work is a finding that a developer can act on — not just a list of things that are broken.

On the job: An engagement ends in a report. The format you learned in the Juice Shop assessment is a format used in production. Clear findings, actionable remediations, evidence-backed.

Secrets management

Hardcoded credentials are a separate vulnerability class from application logic flaws. The fix is not deletion from current code — it is rotation of the exposed credential plus integration of a secret manager (HashiCorp Vault, AWS Secrets Manager) and pre-commit scanning to prevent recurrence.

On the job: Gitleaks runs as a pre-commit hook and a CI gate. TruffleHog runs on a schedule against full git history. When a developer accidentally commits an API key, you have a runbook: alert the developer, rotate the credential immediately (assume it is already compromised), add the pattern to the blocklist. This is day-one operational work at many companies.

V

Cloud Security Fundamentals

Weeks 23–25

Modern AppSec work happens in the cloud. S3 misconfiguration and IAM over-permissioning are among the most frequently reported cloud misconfigurations, and credible threat modeling requires understanding the cloud attack surface an application runs on. This phase covers the AppSec-relevant subset: how to identify misconfigurations in code and architecture reviews, understand SSRF chains that target cloud metadata, and discuss findings with infrastructure teams. It then goes deeper on these subjects: infrastructure as code with Terraform, Kubernetes security, and Azure beside AWS. Its container security step produces a write-up of the misconfigurations in the Phase 0 FastAPI application's Dockerfile and docker-compose.yaml, labelled with its OWASP Top 10 category, A02 Security Misconfiguration.

Learning Resources

- **AWS Security CTF: Attacker Path** — Six levels covering real-world AWS misconfigurations: S3 bucket exposure, credential leakage, IAM misuse, SSRF via the EC2 metadata service, and additional cases. Each level explains the vulnerability and its fix.
- **AWS Security CTF: Attacker & Defender** — Companion to flaws.cloud with two separate paths: attacker path (exploit misconfigurations in Lambda and ECS Fargate) and defender path (incident response on the same app: analyze the attack logs with jq and Athena).
- **AWS Shared Responsibility Model** — Official AWS documentation. Defines what AWS secures (infrastructure) versus what the customer secures (IAM, S3 policies, application code, data encryption).
- **AWS IAM Documentation** — Official AWS documentation. The IAM concepts section covers users, roles, groups, policies, the principle of least privilege, and additional topics.
- **FreeCodeCamp Cloud Security Fundamentals in AWS** — Guide covering IAM users and policies, S3 bucket configurations, MFA, the AWS Shared Responsibility Model, and additional topics, with hands-on examples.
- **Trivy** — Open-source scanner. It finds known vulnerabilities and secrets in a container image. It finds misconfigurations in a Dockerfile. Used to scan the Phase 0 FastAPI image and its Dockerfile.
- **OWASP Top 10** — OWASP's standard awareness document of the most critical security risks to web applications.
- **TryHackMe: Writing Pentest Reports room** — The room that teaches the write-up structure this phase's write-ups use.
- **OWASP Docker Security Cheat Sheet** — A hardening checklist for images and containers: non-root users, pinned base images, capability drops, secrets handling, read-only filesystems.
- **OWASP Kubernetes Security Cheat Sheet** — Read for the security vocabulary: role-based access control (RBAC), network policies, and additional controls.
- **OWASP Infrastructure as Code Security Cheat Sheet** — OWASP's guidance on reviewing and securing infrastructure declared as code.

- **Terraform Tutorials: Get Started** — HashiCorp's official tutorials. The AWS track and the Azure track create, change and destroy real infrastructure from Terraform files. (Azure track)
- **Kubernetes Documentation: Security** — The official security section: Pod Security Standards and RBAC authorization.
- **OWASP Kubernetes Top Ten** — OWASP's ten most common Kubernetes risks, each with a How to prevent section.
- **Kubernetes Goat** — A deliberately vulnerable Kubernetes cluster with scenarios that run on a local cluster.
- **Microsoft Learn: Azure Security Fundamentals** — Official documentation: Azure's security model, role-based access control and managed identities.

Cloud Security Fundamentals — Weeks 23–25

01. READ: AWS Foundations: Shared Responsibility & IAM: Week 23

- Read the AWS Shared Responsibility Model page completely. Write a one-paragraph summary in your own words: what AWS is responsible for, what you are responsible for, and what the boundary looks like for a web application running on EC2 behind an S3 bucket
- Read the IAM concepts in the AWS IAM documentation: users, roles, policies, how they attach, and least privilege in practice
- Read the FreeCodeCamp Cloud Security Fundamentals in AWS guide — hands-on, covers IAM and S3 configuration with real console examples
- **Full notes + note cards:** AWS Shared Responsibility boundary; IAM user vs. role vs. group; what a policy document looks like (JSON structure, Effect/Action/Resource); what least privilege means and why wildcard permissions (s3:*) are a finding

02. LABS: flaws.cloud — Levels 1–6: Week 23

- Set up a free-tier AWS account with the AWS CLI installed and configured. The CTF runs against real AWS infrastructure
- Complete all six levels.
- **After all six levels:** one combined writeup in the write-up structure from Writing Pentest Reports, one finding per level

03. LABS: flaws2.cloud — Attacker & Defender Paths: Week 23

- **Two distinct paths:** attacker (exploit serverless and container misconfigurations) and defender (investigate the same attack from the logs).
- Complete both paths. The defender path is the AppSec-relevant half — it teaches you to read what an attack left behind in the logs, which is how a finding is confirmed on the job
- **Key concepts reinforced:** IAM role scoping for Lambda and ECS tasks, container metadata credential exposure, and log analysis with CloudTrail, jq, and Athena

04. LEARN + PRACTICE: Container Security Basics: Week 23

- Read the OWASP Docker Security Cheat Sheet
- Install Trivy, then scan the image of the Phase 0 FastAPI application for vulnerabilities and secrets, and scan its Dockerfile for misconfigurations
- Read the OWASP Top 10 page for A02 Security Misconfiguration. Then, audit the application's Dockerfile and docker-compose.yaml against the description on that page and the rules of the cheat sheet
- Write up the findings, with the Trivy results as evidence, in the write-up structure from Writing Pentest Reports, and label them A02 Security Misconfiguration
- Fix the findings, rebuild the image, and scan again. Add the fixed code and the new scan results to the write-up
- Fix only these findings. The application's other deliberate weaknesses stay in place
- Read the OWASP Kubernetes Security Cheat Sheet for the security vocabulary: RBAC, network policies, and additional controls
- **Full notes + note cards:** image scanning vs runtime security; least privilege in containers; where secrets must not live (image layers, environment defaults); IMDS access from a container

05. READ + PRACTICE: Infrastructure as Code with Terraform: Week 24

- Read the OWASP Infrastructure as Code Security Cheat Sheet: infrastructure as code means the servers, networks and permissions an application runs on are declared in files and created by a tool, so they can be reviewed like source
- Read What is Infrastructure as Code with Terraform?, then install Terraform
- Work through the AWS track in order: Create infrastructure, Manage infrastructure and Destroy infrastructure. The tutorial creates a real instance in your AWS account; destroy it at the end of every sitting so nothing is billed
- Work through the Azure track in order: Build infrastructure, Change infrastructure, Define input variables, Query data with output values and Destroy infrastructure. The same rule applies to the Azure account
- Read Trivy misconfiguration scanning, then scan the Terraform files you wrote in both tracks and read every finding against the cheat sheet
- Write up the findings from the scan in the write-up structure from Writing Pentest Reports

06. READ + LABS: Kubernetes Security: Weeks 24–25

- Read the Kubernetes security concepts page, then Pod Security Standards and Using RBAC Authorization
- Read the ten entries of the OWASP Kubernetes Top Ten (2025 edition), each with its How to prevent section
- Set up Kubernetes Goat on a local cluster as its setup page describes, then complete its scenarios in the order the docs list them. Each scenario is a deliberately vulnerable deployment; write lab notes for each and name the Top Ten entry it demonstrates
- Write up the scenarios that map to entries K01 to K03 in the write-up structure from Writing Pentest Reports

07. READ: Azure Security Beside AWS: Week 25

- Read the Introduction to Azure security and compare its shared responsibility section with the AWS model from step 01
- Read What is Azure role-based access control and What is managed identities for Azure resources, and map each concept to its AWS IAM counterpart from step 01
- Re-read the Terraform Azure track files from step 05 and mark every place where a role assignment or identity is declared

Projects & Writings — Cloud Security Fundamentals

Writing: flaws.cloud assessment writeup

One combined writeup in the write-up structure from Writing Pentest Reports, covering all six levels, one finding per level.

Writing: Container security write-up

A write-up of the misconfigurations in the Phase 0 FastAPI application's Dockerfile and docker-compose.yaml, with the Trivy results as evidence, the fixed code, and the scan results after the fix, labelled A02 Security Misconfiguration.

Writing: Infrastructure as code write-up

A write-up of the Trivy misconfiguration findings in the Terraform files from the AWS and Azure tracks, in the write-up structure from Writing Pentest Reports.

Writing: Kubernetes Goat write-ups

Write-ups of the Kubernetes Goat scenarios that map to the first three OWASP Kubernetes Top Ten entries, in the write-up structure from Writing Pentest Reports, with lab notes for every scenario.

Concepts and Their Use on the Job

AWS Shared Responsibility Model

AWS secures the physical infrastructure. You secure what runs on it: IAM configurations, S3 policies, application code, data encryption. Misunderstanding this boundary is the root cause of many cloud breaches.

IAM — Identity and Access Management

Users, roles, groups, and policies. A role is what an AWS service or EC2 instance assumes to perform actions. A policy is a JSON document defining what actions are allowed or denied on which resources. Least privilege means a role should have exactly the permissions it needs and nothing more. Over-permissioned roles are a common IAM finding.

On the job: When a team presents a new service design, you ask: what IAM role does this service run as? What permissions does that role have? Could an attacker compromise this service and use its IAM role to access something they should not? Reading an IAM policy JSON, an S3 bucket policy, or a Lambda configuration and spotting the problem is an AppSec-specific cloud skill: auditing infrastructure, not writing it.

S3 bucket misconfiguration

Public read access means anyone on the internet can read the contents. Authenticated AWS user access means any AWS account holder worldwide can read it. Both are common misconfigurations. Bucket policies, ACLs, and Block Public Access settings all interact, and a reviewer has to know which one controls what.

On the job: When reviewing a deployment configuration or an IaC pull request, you check: is Block Public Access enabled at the bucket level? Are there wildcard permissions? Are access logs enabled on buckets containing sensitive data?

EC2 instance metadata service (IMDS)

169.254.169.254 is accessible from within an EC2 instance and returns IAM credentials, instance ID, and configuration. SSRF against this endpoint gives an attacker the instance's IAM credentials. IMDSv2 requires a token header that prevents simple SSRF exploitation. Whether IMDSv2 is enforced is a standard AppSec checklist item.

On the job: When you see an endpoint that accepts a URL and fetches it server-side, your first question is now: can this reach 169.254.169.254? Is IMDSv2 enforced on the EC2 instances this service runs on? Is there a URL allowlist? flaws.cloud Level 5 is the exact attack chain you are looking for.

Credential exposure in git history

AWS keys committed to a git repository — even if deleted in a later commit — are permanently visible in history. TruffleHog scans history. The fix is to rotate the exposed credential, not just delete it from current code, plus pre-commit scanning to prevent recurrence.

On the job: When reviewing any code that configures AWS clients, check: are credentials hardcoded? Are they pulled from environment variables (better but still risky) or from a secrets manager or IAM role (correct)?

VI

Secure Code Review & Threat Modeling

Weeks 26–28

Phase VI builds the two highest-leverage AppSec skills — the ones that appear earliest in a job. Secure code review is the daily work of most AppSec engineers at software companies. Threat modeling is how AppSec influence scales beyond individual code reviews to affect system design. OWASP ASVS is studied here as the verification standard that gives code review a systematic checklist — turning ad-hoc review into structured, repeatable methodology. Its ASVS step produces a write-up of the logging gaps in the Phase 0 FastAPI application, labelled with its OWASP Top 10 category, A09 Security Logging and Alerting Failures.

Learning Resources

- **OWASP Code Review Guide v2.0** — A methodology reference for secure code review.
- **OWASP Application Security Verification Standard (ASVS) v5.0.0** — Approximately 350 testable security requirements covering authentication, access control, input validation, cryptography, API security, configuration, and additional areas. Level 1 is the first layer of defense; Level 2 covers standard security practice.
- ***"Threat Modeling: Designing for Security"*** — Adam Shostack, Wiley, 2014 — Ch. 1–3 cover a first threat model, data flow diagrams, and STRIDE; Ch. 4–5 cover attack trees and attack libraries.
- **GitHub Security Lab** — Vulnerabilities found by Security Lab researchers in open source software, published as advisories.
- **OWASP Threat Dragon** — Threat modeling tool. Used for the threat modeling project in this phase.
- **OWASP Logging Cheat Sheet** — What an application should log, what each log entry records, and what to keep out of logs.
- **Python Logging HOWTO** — The Python documentation's guide to the logging module. Its Basic Logging Tutorial covers loggers, levels, logging to a file, and message format.
- **OWASP Top 10** — OWASP's standard awareness document of the most critical security risks to web applications.
- **TryHackMe: Writing Pentest Reports room** — The room that teaches the write-up structure this phase's write-ups use.

Secure Code Review & Threat Modeling — Weeks 26–28

01. READ + PRACTICE: Secure Code Review Methodology: Weeks 26–27

- Read the OWASP Code Review Guide, Introduction through Methodology, then its Technical Reference (methodology, code review techniques, vulnerability-specific review guidance)
- Conduct a scoped code review of an open-source Python web application (Django, Flask, or FastAPI) from GitHub. Scope it to one subsystem, for example authentication and session handling, roughly 2,000 lines, and budget about 8 hours

- **Apply the systematic methodology:** trace user input, verify authentication, check authorization, audit crypto, check dependencies

02. READ + PROJECT: Threat Modeling: Weeks 26–28

- Read "*Threat Modeling: Designing for Security*" Ch. 1–5
- Build a complete threat model for your Phase 0 FastAPI application, including the LLM endpoint added in Phase IV. Your own layered application is a practical target: you know its layers and can reason about its trust boundaries.
- Use OWASP Threat Dragon for the data flow diagram. Apply STRIDE to each component
- **Produce a threat model document:** DFD, trust boundaries, threat enumeration, risk rating, mitigations
- Write a LinkedIn article from the threat model, 'Threat Modeling a FastAPI Application with STRIDE' (1,000–1,500 words): the data flow diagram, what STRIDE identified, and the mitigations chosen

03. STUDY: OWASP ASVS — Verification Standard: Week 28

- Read the ASVS v5.0.0 Level 1 and Level 2 requirements. Learn the structure and the categories
- Map each ASVS chapter to the vulnerability classes studied in this plan — input validation and output encoding, authentication, access control, and cryptography each get their own chapter, so you can pin any finding to a specific numbered requirement
- Build a personal ASVS checklist: 20–30 of the highest-signal Level 2 requirements you would check first in a code review
- Read the OWASP Top 10 page for A09 Security Logging and Alerting Failures. Then, audit the Phase 0 FastAPI application against the ASVS Security Logging and Error Handling chapter
- Write up the findings in the write-up structure from Writing Pentest Reports, and label them A09 Security Logging and Alerting Failures
- Read the OWASP Logging Cheat Sheet and the Basic Logging Tutorial of the Python documentation. Then, add logging of authentication and authorization events to the application, plus add the fixed code to the write-up
- Give remediation advice (no code) for a finding that cannot be fixed inside the application
- Fix only these findings. The application's other deliberate weaknesses stay in place

04. PRACTICE: Code Review Under Time Pressure: Week 28

- Use GitHub Security Lab's published CVE writeups as answer keys: read the vulnerable code before the writeup, try to identify the vulnerability, then compare. Code review on the job is time-boxed, because a pull request waits on the reviewer
- Do the following five, one per vulnerability class:
  - Cross-Site Scripting (XSS) in the sign-in page of typebot.io
  - Blind server-side request forgery (SSRF) in Medusa
  - SQL injection vulnerability in FarmBot's web app
  - Path traversal in yt-dlp leading to RCE
  - Unsafe deserialization in MkDocs
- **Target:** 30 minutes from cold code to written vulnerability identification

Projects & Writings — Secure Code Review & Threat Modeling

Project: Open-Source Code Review

Documented security review of a real open-source Python application. Findings in standard format, pushed to GitHub. Includes scope, methodology, findings, and remediation recommendations.

Project: Threat Model Document

Full STRIDE threat model of a system built end-to-end in this plan. DFD, trust boundaries, threat enumeration, risk ratings, mitigations. Pushed to GitHub. Shows design-time security thinking, not only exploitation. The target is your Phase 0 FastAPI app including the LLM endpoint added in Phase IV, so STRIDE has to cover prompt injection and output handling as well as the classic web threats.

Checklist: Personal ASVS Review Checklist

20–30 of the highest-signal ASVS Level 2 requirements, ordered as you would check them in a code review. Pushed to GitHub.

Writing: Security logging write-up

A write-up of the logging gaps in the Phase 0 FastAPI application, found by an audit against the ASVS Security Logging and Error Handling chapter, with the fixed code, labelled A09 Security Logging and Alerting Failures.

Writing: ‘Threat Modeling a FastAPI Application with STRIDE’ (1,000–1,500 words)

LinkedIn article drawn from the threat model of the Phase 0 FastAPI application: the data flow diagram, what STRIDE identified, and the mitigations chosen.

Concepts and Their Use on the Job

Secure code review as methodology

Systematic input tracing, authentication verification, authorization checking, cryptography auditing. Replicable and teachable.

On the job: When a developer asks you to review a PR, you apply the methodology: trace inputs, check auth, check authz, check crypto, check dependencies. Faster and repeatable when systematic.

OWASP ASVS as a verification framework

~350 testable security requirements organized by domain. Level 1 is the first layer of defense against common attacks. Level 2 is a comprehensive view of standard security practice. Using ASVS during code review means you can say ‘this fails ASVS V2.1.1’ rather than ‘this seems wrong’ — a finding a developer can look up and act on.

STRIDE threat modeling

Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. A threat category for every component of a data flow diagram.

On the job: When a team presents a new microservice design, you build a DFD on the whiteboard, identify trust boundaries, and run STRIDE. One threat model at design time can prevent many vulnerability findings after code is written.

Trust boundaries

The lines in a DFD where data crosses from less-trusted to more-trusted context. A trust boundary is a potential vulnerability surface.

Time-pressured code review

Cold code, 30–45 minutes, a written or spoken identification of the flaw. The same format as a PR review under deadline.

VII

CompTIA Security+ Certification

Weeks 29–32

This phase is conditional: take it only if you choose work that requires a United States security clearance or a defence-contractor baseline certificate. Postings that require CompTIA Security+ generally also require a clearance or list it among accepted baseline certificates, and the defence manual gives no grace period after hire, so the certificate must be held when the work starts. If that route is not chosen, skip this phase and begin Phase VIII.

Learning Resources

- **"*CompTIA Security+ Get Certified Get Ahead: SY0-701 Study Guide*"** — Joe Shelley and Darril Gibson, Certification Experts LLC, 2023 — Covers all SY0-701 exam objectives across eleven chapters with real-world examples and exam topic review sections.
- **Professor Messer Practice Exams** — Paid practice exams for the Security+ SY0-701 exam. Same format and structure as the actual exam.

CompTIA Security+ Certification — Weeks 29–32

01. READ + MEMORIZE: Study Guide: Weeks 29–32

- Read "*CompTIA Security+ Get Certified Get Ahead: SY0-701 Study Guide*" cover to cover. The exam has five domains: General Security Concepts, Threats/Vulnerabilities/Mitigations, Security Architecture, Security Operations, and Security Program Management & Oversight
- Most technical content from the first three domains will be review from earlier phases. Focus study time on the governance, compliance, and program management domains
- **Full notes, then granular cards:** label and study note cards in book order — one card per concept (title on the front, 1–5 bullets on the back), roughly 160–200 cards, with extra cards where a topic is dense.
- Prioritize the high-yield memorization sets: ports and protocols, cryptography and PKI, attack types, security control types, and the governance, compliance, and framework material
- **Memorize (grouped spaced review):** memorize cards in groups of seven with full front-to-back recall, fold each group into a growing cumulative review pile, and re-drill the whole pile each time you add a new memorized 7 card group to it (pruning only over-learned cards). Budget real time — at ~180 cards this is a multi-day make-and-memorize effort, not an afterthought

02. PRACTICE: Practice Exams: Weeks 31–32

- Take a full Professor Messer practice exam cold. Score it and identify weak domains
- Study weak domains specifically from the study guide via note cards and additional learning, if required. Retake. Repeat until score 85–90% over 2–3 practice exams in a row

03. EXAM: CompTIA Security+ SY0-701: Week 32

- **Delivery:** schedule through Pearson VUE and take it in person at a Philadelphia test center (nearest: Temple University, North Broad Street) — about $439 (2026), valid for three years
- **Scheduling:** book the soonest available seat once you are consistently at 85–90% on practice exams. Specific dates fill, so reserve the nearest one the moment you hit target

Projects & Writings — CompTIA Security+ Certification

Certification: CompTIA Security+ (SY0-701)

Passed exam, valid for three years; the vendor-neutral baseline that confirms shared vocabulary with SOC, infrastructure, and GRC colleagues before AppSec-specific work begins.

Concepts and Their Use on the Job

Security+ exam domains

All five SY0-701 domains: General Security Concepts, Threats/Vulnerabilities/Mitigations, Security Architecture, Security Operations, Security Program Management & Oversight.

On the job: Security+ signals baseline security literacy to enterprise companies and regulated industries, where it is a common requirement. The portfolio carries the hands-on evidence.

Governance and compliance

Risk management frameworks, regulatory compliance (HIPAA, PCI-DSS, SOC 2), security policy and procedure, security awareness training.

On the job: AppSec engineers work with compliance teams. Knowing what PCI-DSS requires for web applications, what HIPAA means for data handling, and how risk frameworks are structured makes cross-team conversations productive.

Cryptography fundamentals

Symmetric and asymmetric encryption, hashing algorithms, PKI, certificate lifecycle, TLS/SSL configuration.

Network security

Firewalls, IDS/IPS, VPN, network segmentation, wireless security.

Identity and access management

Authentication protocols, MFA, SSO, directory services, privileged access management.

Incident response

IR lifecycle phases, forensics fundamentals, log analysis, chain of custody.

Shared vocabulary with the wider security team

Cryptography, network security, and authentication fundamentals from Security+ are the common ground with SOC, infrastructure, and GRC colleagues. The certification confirms that baseline before AppSec-specific work begins.

VIII

Interview Preparation & Interviewing

Weeks 33–36

AppSec interviews test security knowledge, adversarial reasoning, and communication. This phase runs interview preparation alongside the ongoing job search: rehearsing behavioral answers, drilling technical note cards, practicing the live technical formats — secure code review, light scripting, and system design — and running mock interviews.

Learning Resources

- **"*Impractical Python Projects*"** — Lee Vaughan, No Starch Press, 2018 — Project-based Python practice for short scripting exercises
- **GitHub Security Lab** — Vulnerabilities found by Security Lab researchers in open source software, published as advisories.
- **Security interview question bank** — A curated set of cybersecurity interview questions to build a technical note card deck from
- **Mock interview platforms** — Aced (formerly Exponent) (security engineering courses) and interviewing.io (behavioral and observed coding practice)

Interview Preparation & Interviewing — Weeks 33–36

01. APPLY: Main Application Push: Week 33 onward

- Begin the main application push, following "Application Strategy" in the Appendix
- **Daily target:** 1–2 applications a day, each with a cover letter tailored to the specific role and the resume tailored as needed — budget roughly 1–2 hours per application

02. REHEARSE: Behavioral Questions: Weeks 33–36

- Write out full answers to the standard behavioral questions and rehearse them aloud.
- Rehearse AppSec-specific prompts as well: handling a developer who disputes a finding, prioritizing a backlog of vulnerabilities, explaining risk to a non-technical stakeholder, and so forth.

03. MEMORIZE: Technical Note Cards: Weeks 33–36

- Build note cards organized by topic: the OWASP Top 10 and the individual web vulnerability classes, the Security+ domains, PortSwigger attack techniques, and defensive and secure-coding patterns.
- Alongside cards drawn from your own study, create a deck from a reputable existing set of cybersecurity interview questions at a curated public question bank, rewriting the answers in your own words.
- Use the same grouped spaced review as Phase VII step 01: groups of seven, cumulative review pile, periodic re-reads of finished topics.

04. PRACTICE: Secure Code Review: Weeks 33–36

- Practice a clear, spoken routine until it becomes automatic: read the code, name the vulnerability class, explain how it is exploited, and recommend the fix.
- Work through the following published CVE write-ups from GitHub Security Lab on unfamiliar code, at about five reviews each week:
  - Cross-Site Scripting (XSS), Unvalidated Redirect, and Extensive Remote Proxy Capabilities in NocoDB
  - Cross-site scripting (XSS) in OpenLibrary barcode scanner
  - Reflected Cross-Site Scripting (XSS) vulnerabilities in habitica
  - Remote Code Execution (RCE) via Cross-Site Scripting (XSS) in Umbrel
  - Cross-site scripting (XSS) in Common Voice
  - Server-Side Request Forgery (SSRF) and a weak default CORS configuration in rembg
  - Blind server-side request forgery (SSRF) vulnerabilities in Audiobookshelf
  - Server-Side Request Forgery (SSRF) and Denial of Service (DoS) in Mealie
  - Server-Side Request Forgery (SSRF) in MouseTooltipTranslator
  - SQL injection vulnerability in Meshery
  - SQL injection vulnerability in TaxonWorks
  - Path traversal in AWS SAM CLI
  - Path injection, Cross-Site Scripting (XSS) and CORS misconfiguration in Flowise
  - Command injection in GPT-SoVITS
  - Authenticated RCE in binance-trading-bot via command injection
  - Remote Code Execution (RCE) in GPT-SoVITS
  - Authorization Issues in Outline
  - Unauthorized access to event data across organizational boundaries in Sentry
  - Authentication bypass in Rocket.Chat
  - Client-Side Request Forgery (CSRF) in the Home Assistant Companion for iOS and macOS
- When the list is finished and the job search is still running, continue at the same rate with write-ups whose titles name a web vulnerability class from Phase III.
- Practice naming the vulnerability class immediately from a code sample — for example: user input passed into a query without sanitizing it (injection); a React component using dangerouslySetInnerHTML (cross-site scripting); a JSON Web Token whose signature algorithm is accepted as ‘none’ (authentication bypass); a user identifier read from a request parameter with no authorization check (insecure direct object reference); or a URL fetched by the server without validation (server-side request forgery).

05. PRACTICE: Light Scripting & System Design: Weeks 33–36

- Practice short Python scripting without IDE hints; some companies include a short coding session (ex. CoderPad): a small script to parse a log, fix a function, or automate a check. *Impractical Python Projects* is good practice.
- Practice the security system-design format: sketch a small architecture — for example the trust boundaries of an authentication flow or a microservice — mark where data crosses each trust boundary,

apply the STRIDE model to enumerate the threats, and rehearse walking through it aloud. This is verbal whiteboard practice, not a written deliverable.

06. **PRACTICE: Mock Interviews:** Weeks 33–36

- Run at least one full mock interview each week, covering behavioral questions and a code review, through Aced (formerly Exponent) (its security engineering courses) or interviewing.io.
- Build up on your own first — untimed, then timed, then recorded and reviewed against your own answers — before booking the weekly mock interviews above.
- When you do not immediately know an answer, narrate your reasoning aloud and state what you do know and how you would investigate it.
- Prepare for the formats that published application security interview loops often use: code review, threat modeling and design conversation. Those loops rarely contain algorithm puzzle rounds, so none is practised here

IX

Expected First Days on the Job

Week 37+

The first weeks in a new role go to internal tooling, company-specific security policies, and how the development teams work.

First Two Weeks

Read the existing security documentation the way you read a codebase — understand what is there before suggesting what should change. Ask about the current vulnerability backlog before proposing new tooling. Understand what development teams build and how they ship.

- What is the current SAST setup, if any? What do developers receive as output?
- What does the existing pentest methodology look like? How are findings tracked?
- Who are the key development team contacts? What are their current security concerns?
- What does the incident response process look like? Has it been tested?

> The first-week question is not what to add but what is already here and why it is the way it is. The answer shows the organizational constraints you are working within.

What You Bring on Day One

Developer empathy

You understand how vulnerable patterns get written, from years of shipping software. That shared experience makes conversations with development teams direct and credible.

API security depth

REST API security is a central AppSec domain. Years of building REST APIs, plus the API security work in this plan, carry over directly.

Automation instinct

Semgrep rule development, CI/CD integration, and tooling scripting reuse skills from engineering work.

Full-stack deployment

You can handle the end-to-end deployment of security tooling: writing the Semgrep rule, integrating it into GitHub Actions, and triaging the results.

Appendix

Reference Material · Application Strategy · Lab Notes Format · Philadelphia AppSec Community

Application Strategy

- **Start applying in Phase IV:** BSCP preparation is under way, and the writeups from Phases I–III are published.
- **Target titles:** Application Security Engineer, Security Engineer I or II, Product Security Engineer, DevSecOps Engineer, and additional related titles. Five years of production development plus this portfolio is relevant experience for postings that ask for security experience.
- **Pin your AppSec work:** set your GitHub pinned repositories to the key new artifacts — the Juice Shop assessment report, your custom Semgrep rule, and the threat model.
- **Make LinkedIn application-ready:** list BSCP and Security+ under licenses and certifications as each is passed; link your portfolio projects and published write-ups; and state the move into application security in the About section. Publish as you work through the plan.
- **Cover letters and interviews:** lead with five years of full-stack software engineering, then the move into application security. Point to this plan and portfolio as the evidence.

Lab Notes Format

Lab notes record how a lab was solved, so the method can be recalled later without solving the lab again.

- **When to write them:** write lab notes for a lab that has none. When a later step assigns a lab that already has notes, update them if the repeat exposed a gap.
- **Title:** the lab's title and URL.
- **Recognition:** what in the application's behavior showed that the vulnerability was present.
- **Exploit:** the working payload or request sequence.
- **Root cause:** why the application was exploitable.
- **Fix:** the change to the code or configuration that removes the vulnerability.
- **Where to keep them:** one searchable file, grouped by vulnerability class. In Phase IV this file becomes a reference for the BSCP exam.

Philadelphia AppSec Community

Security networking operates differently from general tech networking. The community is smaller and more tight-knit. Attending events builds relationships with local practitioners. Show up, ask good questions, and bring something to demonstrate.

Core Groups

- **OWASP Philadelphia Chapter** (events on Meetup · Free) — An AppSec-specific group in Philadelphia. OWASP (Open Web Application Security Project) is a nonprofit foundation for web application security. The Philadelphia chapter holds regular meetings with technical presentations on vulnerability classes, tooling, and real-world AppSec work. The people in this room are AppSec practitioners at companies across the region.
- **DC215: Philadelphia DEF CON Group** (events on Meetup · Free) — DEF CON Groups are local offshoots of DEF CON, an annual hacker conference. DC215 holds regular meetups with technical talks, CTF practice, and informal knowledge sharing. The format rewards people who bring something to show.
- **BSides Philadelphia** (Annual · December 11, 2026 · $80 early bird) — Community-organized security conference with talks across offensive, defensive, and AppSec tracks. Affordable, full-day, attended by local practitioners. Volunteer if possible.
- **Philly 2600** (First Friday monthly · Free) — Philadelphia's longest-running hacker meetup — running since the early 1990s and one of the first 2600 meetings listed in 2600 Magazine. Meets the first Friday of every month at Iffy Books starting at 6pm. No set agenda, no presentations, no gatekeeping — a casual hangout where people bring things to hack on or show off. Open to anyone. Around 8pm the group typically moves to a nearby bar. A good place to show up early in the plan.
- **PhillySec** (Second Wednesday monthly · Free) — Monthly security meetup, second Wednesday of each month. More structured than Philly 2600 — presentations and technical discussions rather than open hangout format. A good mid-plan target.
- **OWASP Delaware Chapter** (Wilmington, DE · Monthly · Free) — The OWASP chapter an hour south. Runs a monthly MetaCTF flash CTF night, hands-on exercises, and secure-coding presentations. A second AppSec-specific room when the Philadelphia calendar is quiet.
- **Ex Machina Parlor** (319 N 11th St, Room 5D · Philadelphia) — A hackerspace specifically aimed at people interested in cybersecurity. Workshop and collaborative hacking space rather than a meetup. Worth knowing it exists as a physical space to work alongside practitioners. Check their calendar for events and open nights.

Conferences

- **OWASP Global AppSec USA** (2026: November 2 to 6, San Francisco) — OWASP's largest US conference. This conference matches this plan closely: the audience is AppSec practitioners and the talks map to the skills in this plan. Attend once Phase IV or V is complete.
- **OWASP BASC (Boston)** (2027: Saturday, April 3, one day, Cambridge, MA) — The OWASP Boston Application Security Conference. Regional, focused on AppSec engineering and secure development. Lower-cost than Global AppSec USA. Strong emphasis on hands-on content and OWASP standards — the audience overlaps heavily with who you will work alongside in the role.

- **Thotcon (Chicago)** (next event June 11–12, 2027) — A mid-sized hacker conference: more intimate than DEF CON, technically dense, practitioner-focused.
- **JawnCon (Glenside, PA)** (Annual · October) — Philadelphia's own security and technology conference, held at Arcadia University just outside the city. Covers a wide range of security and tech topics with a deliberately approachable, community feel. Worth attending as a local conference where you will run into the same practitioners from OWASP and DC215.
- **BSides Delaware (Newark, DE)** (Annual · November 13–14, 2026 · $25) — The nearest BSides after Philadelphia, at the University of Delaware, about an hour away. Talks, villages, and a CFP; the same regional practitioner crowd as BSides Philly.
- **DEF CON (Las Vegas)** (Annual · August) — An annual hacker conference. Within it, the AppSec Village is a 100% volunteer-run space specifically for application security — attacker-minded talks, live demos, hands-on workshops, and an AppSec-focused CTF. A larger, costlier trip than the regional events.